

LESS: Selecting Influential Data for Targeted Instruction Tuning

Mengzhou Xia^{1*} Sadhika Malladi^{1*} Suchin Gururangan² Sanjeev Arora¹ Danqi Chen¹

Abstract

Instruction tuning has unlocked powerful capabilities in large language models (LLMs), using combined datasets to develop general-purpose chatbots. However, real-world applications often require a specialized suite of skills (e.g., reasoning). The challenge lies in identifying the most relevant data from these extensive datasets to effectively develop specific capabilities, a setting we frame as *targeted instruction tuning*. We propose LESS, an optimizer-aware and practically efficient algorithm to estimate data influences and perform Low-rank gradiEnt Similarity Search for instruction data selection. Crucially, LESS adapts existing influence formulations to work with the Adam optimizer and variable-length instruction data. LESS first constructs a highly reusable and transferable *gradient datastore* with low-dimensional gradient features and then selects examples based on their similarity to few-shot examples embodying a specific capability. Experiments show that training on a LESS-selected 5% of the data can often outperform training on the full dataset across diverse downstream tasks. Furthermore, the selected data is highly transferable: smaller models can be leveraged to select useful data for larger models and models from different families. Our qualitative analysis shows that our method goes beyond surface form cues to identify data that exemplifies the necessary reasoning skills for the intended downstream application. To facilitate future work, we release code and data at princeton-nlp/LESS.

^{*}Equal contribution ¹Princeton Language and Intelligence (PLI), Princeton University, USA ²Department of Computer Science, University of Washington, USA. Correspondence to: Mengzhou Xia, Sadhika Malladi <mengzhou, smalladi@princeton.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

1. Introduction

prompt-response

Instruction tuning has made large language models (LLMs) adept at following human instructions (Ouyang et al., 2022) as versatile chatbots (OpenAI, 2022; 2023; Anthropic, 2023; Google, 2023). Recent efforts curating highly diverse and wide-ranging instruction tuning datasets (Taori et al., 2023; Wang et al.; Mukherjee et al., 2023; Xu et al., 2023, *inter alia*) induce remarkably strong generalization even from a small number of examples (Zhou et al., 2023). Regardless, it remains an open problem to understand how to best utilize these various datasets.

Many real-world applications call for cultivating a specific suite of capabilities in LLMs (e.g., reasoning skills). However, training LLMs with mixed instruction tuning datasets can hinder the development of these specific capabilities. For example, Wang et al. (2023b) demonstrates that LLMs trained on a mix of instruction tuning datasets exhibit worse performance than those trained on a subset of the data. Additionally, considering the broad spectrum of user queries and the multitude of skills required to respond to them, there may not always be enough in-domain data available. Therefore, we hope to be able to effectively use the general instruction tuning data to improve specific capabilities. We frame this setting as *targeted instruction tuning*:

Given just a handful of examples embodying a specific capability, how can we effectively select relevant fine-tuning data from a large collection of instruction datasets?

We approach this problem by prioritizing training on data that directly minimizes loss on a target task instead of relying on surface form features (Gururangan et al., 2020; Xie et al., 2023b). Inspired by past works estimating the influence of individual training datapoints with gradient information (Pruthi et al., 2020; Han et al., 2023), we design an optimizer-aware approach to select such data. However, straightforward application of this influence formulation faces several challenges unique to the instruction tuning setting: (1) LLMs are traditionally fine-tuned with the Adam optimizer (Kingma & Ba, 2015) instead of the canonical SGD optimizer; (2) using sequence-level gradients of variable-length instruction data can derail the influence estimation; and (3) the large number of trainable parameters in LLMs makes the computation and storage of gradient information extremely resource-intensive.

We address these concerns in LESS, an algorithm that performs Low-rank gradiEnt Similarity Search to select relevant instruction tuning data for a target application, which exhibits the following properties:

1. **Compatible with instruction tuning with Adam (§2 and §3):** LESS adapts the gradient features from classical influence formulations (Pruthi et al., 2020) to work with the Adam optimizer and variable-length instruction data. The optimization insights and influence formulation may be of independent interest as well.
2. **Efficient (§4.1):** LESS uses LoRA (Hu et al., 2021) and random projections (Johnson & Lindenstrauss, 1984) to construct a *gradient datastore* with low-dimensional, easily manipulable gradient features that permit efficient and effective dataset selection. The gradient datastore can be reused for new target tasks.
3. **Transferable (§5.3):** Data selected using small models’ gradient features induce strong performance in large models and models from different families, adding to the efficiency of LESS (Table 2).
4. **Interpretable (§6.2):** Qualitative analysis shows that LESS selects data with similar reasoning and skill types as the target task, whereas existing approaches often select data based on surface form cues (e.g., language or topic).

We evaluate our approach on three diverse downstream datasets—MMLU (Hendrycks et al., 2020), TY-DIQA (Clark et al., 2020), and BBH (Suzgun et al., 2023)—each containing distinct subtasks that effectively simulate targeted instruction tuning scenarios. Results show that LESS often selects a small subset of the data (5%) that outperforms training on the full dataset, and the selected subset remains universally effective across model scales and families (Table 2). Comparisons with other data selection methods show that LESS is the *only* consistently effective approach, justifying its relatively high computational cost.

2. Preliminaries: Influence Formulation

We restate the approach in Pruthi et al. (2020) of using a first-order approximation of training dynamics to estimate the influence of a training datapoint on held-out data.¹

Per-step influence. Consider a model θ^t at time step t trained on the loss $\ell(\cdot; \theta^t)$. We can write the first-order Taylor expansion of the loss on a validation datapoint z' as

$$\ell(z'; \theta^{t+1}) \approx \ell(z'; \theta^t) + \langle \nabla \ell(z'; \theta^t), \theta^{t+1} - \theta^t \rangle$$

¹We discuss in Section 7 how this formulation of influence is distinct from influence functions (Koh & Liang, 2017).

For ease of exposition, assume that we are training the model with SGD with batch size 1 and learning rate η_t .² If z is the training data at time step t , we can write the SGD update as $\theta^{t+1} - \theta^t = -\eta_t \nabla \ell(z; \theta^t)$. Then, the Taylor expansion can be written as

$$\ell(z'; \theta^{t+1}) - \ell(z'; \theta^t) \approx -\eta_t \langle \nabla \ell(z; \theta^t), \nabla \ell(z'; \theta^t) \rangle$$

Trajectory influence. The influence of z over the entire training run can be measured by aggregating the influence at every training step that uses z . Since z is used once per epoch, it is natural to express this as a summation over epochs:

$$\text{Inf}_{\text{SGD}}(z, z') \triangleq \sum_{i=1}^N \eta_i \langle \nabla \ell(z'; \theta_i), \nabla \ell(z; \theta_i) \rangle \quad (1)$$

where η_i is the learning rate used during the i th epoch out of N total training epochs and θ_i is the model after the i th epoch of training.

Data selection with influence. While Pruthi et al. (2020) used this insight to identify mislabeled training data, we instead apply this formula to design a data selection strategy. In particular, at each time step t , selecting z to maximize $\langle \nabla \ell(z'; \theta^t), \nabla \ell(z; \theta^t) \rangle$ will drive a larger decrease in the loss on the validation point z' . However, when computing Inf_{SGD} across several epochs, we note that the model checkpoints $\{\theta_i\}$ after the first epoch will depend on the dataset selected for training. This causes the data selection problem to become circular, and we empirically circumvent this problem with a short *warmup training* run on a randomly selected $\mathcal{D}_{\text{warmup}} \subset \mathcal{D}$ for $N = 4$ epochs (see §4.1). Overall, this data selection strategy is especially useful in the transfer learning setting, because it does not require any specific relationship between z' and z . The next two sections describe how we adapt this basic approach to operate efficiently and effectively with instruction tuning.

3. LESS: Estimating the Influence of Instructions

Here, we describe how LESS adapts the influence formulation in Equation (1) to select instructions that effectively induce a target capability. We consider two main conceptual challenges here: (1) past works described in Section 2 focus on the SGD optimizer, but LLMs are traditionally tuned using Adam (§3.1), and (2) instruction tuning exhibits abnormal gradients that cause the prior formulation to heavily favor selecting shorter instructions, thereby hurting performance (§3.2).

²We discuss learning rate schedules and batches in Appendix E.

Notation. We follow the standard autoregressive setting for instruction tuning, where a datapoint z consists of an instruction s and a completion c .³ For each data point, the loss $\ell(z; \theta)$ is the cross entropy between the model distribution $p(\cdot|s)$ and the true label c . When c contains multiple tokens, we use an average of the token-wise cross-entropies in the completion as the loss of z . We use θ^t to indicate the model at the t -th step, and we use θ_i to indicate the model after the i th epoch of training.

3.1. Extension to Adam

The formulation in Equation (1) is unique to optimizing models with SGD. However, instruction tuning is usually performed using the Adam optimizer (Kingma & Ba, 2015).⁴ In this case, the parameter update at a given step is:

$$\begin{aligned} \theta^{t+1} - \theta^t &= -\eta_t \Gamma(z, \theta^t) \\ \text{Adam} \hookrightarrow \Gamma(z, \theta^t) &\triangleq \frac{m^{t+1}}{\sqrt{v^{t+1} + \epsilon}} \\ m^{t+1} &= (\beta_1 m^t + (1 - \beta_1) \nabla \ell(z; \theta^t)) / (1 - \beta_1^t) \\ v^{t+1} &= (\beta_2 v^t + (1 - \beta_2) \nabla \ell(z; \theta^t)^2) / (1 - \beta_2^t) \end{aligned}$$

where all operations are performed elementwise, with β_1 and β_2 as the hyperparameters for the first and second moments, respectively, and ϵ as a small constant. Then, the first-order expansion for the Adam dynamics suggests we should choose z to maximize $\langle \nabla \ell(z'; \theta^t), \Gamma(z, \theta^t) \rangle$. Note that extending the data selection strategy to Adam exacerbates the aforementioned circularity of the procedure, because computing $\Gamma(z, \theta)$ requires accessing the m and v terms, which are determined by prior training gradients. As before, we obtain these from the warmup training (§4.1).⁵

3.2. Addressing Abnormalities in Sequence-Level Gradients

A faithful application of Section 2 to the instruction tuning setting would consider that the gradient of an example z is the average of the gradients of each token in the completion. In particular, we would choose which tokens to train on within each sequence in order to best reduce the validation loss, and notably, optimizing the inner product at a fine-grained level may lead to better results. However, estimating individual token influences is extremely expensive because it requires computing the gradient of a vector of token losses with respect to the model parameters, so we instead perform data selection at the sequence level.

³Some datasets contain multi-turn instructions, but we denote them as single turn instructions for ease of exposition.

⁴Appendix D.1 demonstrates instruction tuning with SGD significantly hurts performance.

⁵We ablate against using $\text{sign}(\nabla \ell(\cdot; \theta_i))$ as an approximation for $\Gamma(z, \theta_i)$, as suggested in past theoretical works (Malladi et al., 2022; 2023). See Appendix D.2.

Because the example gradient is the average of many token gradients, the gradient norm of the sequence $\|\nabla \ell(z; \theta^t)\|$ is negatively correlated with the length of the completion (Figures 3 and 4). This is a well-known issue for influence formulations that compute averaged token gradients for sequences of varying lengths. We discuss this observation further in Appendix H but defer an in-depth study to future work. This phenomenon causes the influence in Equation (1) to heavily upweight shorter training sequences, which in turn leads to markedly worse performance (see Table 13). To mitigate this problem, we normalize the gradient features in LESS and use the cosine similarity instead of the dot product to estimate influences.

Definition 3.1 (Adam Influence). Suppose the model is trained for N epochs, where $\bar{\eta}_i$ is the average learning rate in the i th epoch and θ_i is the model checkpoint after the i th epoch. We define the influence of a training datapoint z on a validation datapoint z' when training with Adam as

$$\text{Inf}_{\text{Adam}}(z, z') \triangleq \sum_{i=1}^N \bar{\eta}_i \cos(\nabla \ell(z'; \theta_i), \Gamma(z, \theta_i))$$

where $\cos$ computes the cosine similarity of the two vectors.

4. LESS: Estimating Influences Efficiently

We have established how to estimate the influence using model gradients, but given the sheer size of LLMs, computing and storing model gradients is still prohibitively expensive. In this section, we describe the design choices that make LESS computationally efficient, and illustrate the computing steps in Figure 1.

We use LoRA (Hu et al., 2021) to efficiently perform the warmup training. Then, we construct a gradient datastore of projected low-dimensional gradient features (Johnson & Lindenstrauss, 1984; Park et al., 2023) that can be reused for different target tasks. Finally, we detail how our data selection algorithm uses the datastore to construct a training dataset that accommodates substructures (e.g., individual reasoning skills) in the target validation set.

Setting. Our goal is to select a subset $\mathcal{D}_{\text{train}}$ of a large, diverse instruction tuning dataset $\mathcal{D}$ such that training a model on $\mathcal{D}_{\text{train}}$ achieves lower loss on a target validation dataset $\mathcal{D}_{\text{val}}$, and therefore performs well on the test data. $\mathcal{D}_{\text{val}}$ can contain several subtasks $\mathcal{D}_{\text{val}}^{(1)}, \dots, \mathcal{D}_{\text{val}}^{(m)}$ (e.g., question answering for multiple languages).⁶ The validation set is fixed and only contain a few examples per subtask (e.g., 1 example per language in TYDIQA). We use a held-out test set $\mathcal{D}_{\text{test}}$ to evaluate the performance of the model trained

⁶This is case for datasets we study in this paper, but our method also applies to datasets with no explicit substructures (i.e., $m = 1$).

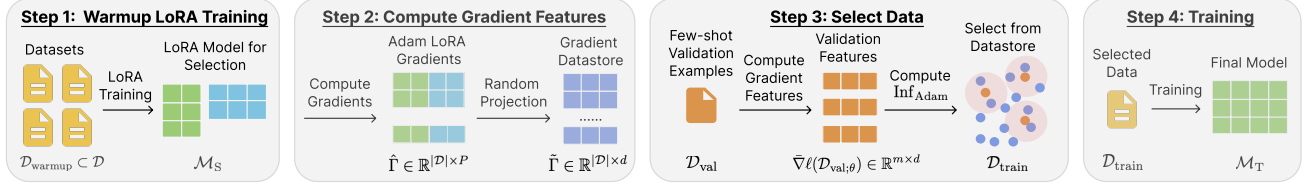


Figure 1: Illustration of LESS. In step 1, we train a selection model $\mathcal{M}_S$ with LoRA for a warmup period with a small subset of data $\mathcal{D}_{\text{warmup}} \subset \mathcal{D}$. In step 2, we compute the Adam LoRA gradient features $\Gamma \in \mathbb{R}^{|\mathcal{D}| \times P}$ for each candidate datapoint and save them in a gradient datastore. In step 3, for any task with few-shot examples $\mathcal{D}_{\text{val}}$ (comprising of m subtasks), we compute the gradient features for each validation subtask and select the subset $\mathcal{D}_{\text{train}}$ with the top 5% training examples ranked by Inf_{Adam} . Step 4 is the final training stage with the selected data on a target model $\mathcal{M}_T$, which can be trained with either LoRA or full finetuning. Steps 1 and 2 are offline and only need to be computed once per candidate training set $\mathcal{D}$.

on $\mathcal{D}_{\text{train}}$. We use $\mathcal{M}_S$ to denote the model parametrized by θ used for scoring and selecting data, and we use $\mathcal{M}_T$ to denote the model trained on the selected data. The selection model $\mathcal{M}_S$ need not be the same as the target model $\mathcal{M}_T$; In this case, we refer to the setting as LESS-T.

4.1. Efficient Gradient Feature Computation

Computing Definition 3.1 requires taking the inner product of two vectors with a size of trainable parameters in the model. Directly using such high-dimensional gradient vectors as features for dataset selection is very computationally expensive, so we apply two techniques to construct meaningful low-dimensional gradient features: parameter efficient fine-tuning via LoRA (Hu et al., 2021) and random projections (Johnson & Lindenstrauss, 1984; Park et al., 2023). We apply these techniques to $\nabla \ell(z'; \theta)$ for validation datapoints z' and to $\Gamma(z, \theta)$ for training datapoints z . To enable rapid transfer to new target validation tasks, we create a datastore with these low-dimensional features for candidate training datapoints.

Step 1: Warmup training with LoRA. We use LoRA (Hu et al., 2021) to reduce the number of trainable parameters and accelerate the inner products in Definition 3.1. LoRA freezes the pre-trained weights and adds a low-rank adaptor to linear layers throughout the network. We use LoRA to instruction tune a pre-trained base model (e.g., LLAMA-2-7B) on a random subset $\mathcal{D}_{\text{warmup}} \subset \mathcal{D}$ for N epochs (we only use 5% of the training data in practice, see §5.1), checkpointing the model after each epoch to store $\{\theta_i\}_{i=1}^N$. The gradient when training with LoRA, denoted $\tilde{\nabla} \ell(\cdot; \theta) \in \mathbb{R}^P$, is much lower dimensional than the model itself; for example, in LLAMA-2-7B, $\tilde{\nabla} \ell(\cdot; \theta)$ is less than 2% the size of θ . We use $\tilde{\nabla} \ell(\cdot; \theta)$ to compute the Adam update and denote it as $\hat{\Gamma}(\cdot, \theta)$. This initial warmup training is motivated conceptually in §3.1, and empirical results in §6.1 demonstrate that omitting it yields suboptimal results.

Step 2: Projecting the gradients. To further reduce the feature dimensionality, we apply a random projection to the LoRA gradients. The Johnson-Lindenstrauss Lemma (Johnson & Lindenstrauss, 1984) asserts that such projections often preserve the inner products in Definition 3.1, thereby ensuring these low-dimensional gradient features are still useful for dataset selection. For a given validation datapoint z' and model checkpoint θ_i , we can compute a d -dimensional projection of the LoRA gradient $\tilde{\nabla} \ell(z'; \theta_i) = \Pi^\top \hat{\nabla} \ell(z'; \theta_i)$, with each entry of $\Pi \in \mathbb{R}^{P \times d}$ drawn from a Rademacher distribution (i.e., $\Pi_{ij} \sim \mathcal{U}(\{-1, 1\})$). For training datapoints z , we compute $\tilde{\Gamma}(z, \cdot) = \Pi^\top \hat{\Gamma}(z, \cdot)$.

We use the memory-efficient online implementation of random projections from Park et al. (2023) to compute and apply Π . In practice, we choose $d = 8192$.

4.2. Data Selection Algorithm

The data selection strategy (see step 3 in Figure 1) adapts the approach in Section 2 to efficiently operate on subtasks of the validation set (e.g., individual tasks within BBH). For each subtask $\mathcal{D}_{\text{val}}^{(j)}$, we compute its average gradient feature for every model checkpoint $\theta_1, \dots, \theta_N$:

$$\bar{\nabla} \ell(\mathcal{D}_{\text{val}}^{(j)}; \theta_i) = \frac{1}{|\mathcal{D}_{\text{val}}^{(j)}|} \sum_{z' \in \mathcal{D}_{\text{val}}^{(j)}} \tilde{\nabla} \ell(z'; \theta_i).$$

As in Definition 3.1, we then aggregate the scores of how close the given datapoint is to each validation subtask throughout training. For example, we can adapt Definition 3.1 to write

$$\text{Inf}_{\text{Adam}}(z, \mathcal{D}_{\text{val}}^{(j)}) = \sum_{i=1}^N \tilde{\eta}_i \frac{\langle \bar{\nabla} \ell(\mathcal{D}_{\text{val}}^{(j)}; \theta_i), \tilde{\Gamma}(z, \theta_i) \rangle}{\|\bar{\nabla} \ell(\mathcal{D}_{\text{val}}^{(j)}; \theta_i)\| \|\tilde{\Gamma}(z, \theta_i)\|}. \quad (2)$$

We select training datapoints that can improve performance on any one of the validation subtasks. Following the logic in §2, we compute the score for z as the maximum across all subtasks: $\max_j \text{Inf}_{\text{Adam}}(z, \mathcal{D}_{\text{val}}^{(j)})$. We select the highest

$$\text{Score} = \max(\rho_1, \rho_2, \dots)$$

scoring examples to construct $\mathcal{D}_{\text{train}}$.⁷ After selection, we use the selected subset $\mathcal{D}_{\text{train}}$ to train the target model $\mathcal{M}_T$.

5. Experiments

We introduce the experimental setup (§5.1) and relevant baselines (§5.2), and then we present the main results (§5.3).

5.1. Experimental Setup

Training datasets. We follow (Wang et al., 2023b) and use the following instruction tuning datasets: (1) datasets created from existing ones such as FLAN V2 (Longpre et al., 2023) and CoT (Wei et al., 2022c); (2) open-ended generation datasets with human-written answers including DOLLY (Conover et al., 2023) and OPEN ASSISTANT 1 (Köpf et al., 2023). The datasets, comprising approximately 270K data points, vary widely in their format and underlying reasoning tasks, and do not contain any obvious in-domain data for the target queries. Appendix A.1 contains more dataset details.

Table 1: Statistics of evaluation datasets. The selection of evaluation tasks cover different kinds of answer types.

Dataset	# Shot	# Tasks	$ \mathcal{D}_{\text{val}} $	$ \mathcal{D}_{\text{test}} $	Answer Type
MMLU	5	57	285	18,721	Letter options
TYDIQA	1	9	9	1,713	Span
BBH	3	23	69	920	COT and answer

Evaluation datasets. We evaluate our method on MMLU (Hendrycks et al., 2020), TYDIQA (Clark et al., 2020) and BBH (Suzgun et al., 2023). MMLU consists of multiple-choice questions spanning 57 tasks including elementary mathematics, US history, computer science, law, and more. TYDIQA is a multilingual question-answer dataset containing 9 typologically diverse languages. Given a question and a related passage, the task requires extrating the answer from the passage. BBH is a collection of 27 challenging tasks from BIG-Bench selected to evaluate reasoning capabilities. Table 1 contains more details about these tasks. Each dataset includes multiple subtasks, and each subtask comes with few-shot examples. These examples are used as $\mathcal{D}_{\text{val}}$ for data selection (§4.2) and as few-shot in-context learning demonstrations in evaluation. Appendix B contains more dataset details.

Models for data selection and training. We test LESS with three base models: LLAMA-2-7B (Touvron et al., 2023), LLAMA-2-13B (Touvron et al., 2023), and MISTRAL-7B (Jiang et al., 2023). In the transfer setting LESS-T, we select using LLAMA-2-7B as $\mathcal{M}_S$ and train

⁷We ablate against the analogous form of the algorithm using InfSGD in Equation (1) for data selection. See Appendix D.2.

either LLAMA-2-13B or MISTRAL-7B as the target model $\mathcal{M}_T$. The warmup training and the final model training are both conducted with LoRA (Hu et al., 2021). We report the average performance and standard deviation across three random seeds. Appendix A.2 contains more training details.

Default setting. LESS performs warmup training on a randomly selected 5% of the complete dataset $\mathcal{D}_{\text{warmup}}$ for $N = 4$ epochs and computes 8192-dimensional gradient features on the data $\mathcal{D}$ (§4.1). For each target task, we use these features to score datapoints by their influences (Definition 3.1) and select the highest scored 5% of $\mathcal{D}$ to construct $\mathcal{D}_{\text{train}}$. We train the target model $\mathcal{M}_T$ on this selected data $\mathcal{D}_{\text{train}}$.

5.2. Baselines

We compare LESS with a several baselines (see more details in Appendix C). The simplest baseline is random selection, where we randomly sample data from the training set for instruction tuning. We also compare to BM25 (Robertson et al., 2009), which featurizes examples by their word frequency statistics (i.e., TF-IDF) to rank the training instances, and select the top k data points with the highest scores to construct $\mathcal{D}_{\text{train}}$. Another baseline is DSIR (Xie et al., 2023b), which uses n -gram features to weight candidate training data $\mathcal{D}$. We resample k data points according to the importance weights. We also compare to RDS (Representation-based Data Selection) (Zhang et al., 2018; Hanawa et al., 2020), which uses model’s hidden representations as features for data selection. For fair comparison, we compute the similarity score using Equation (2) but replace the gradient features with the final layer representations of the last token of each sequence. These features are 2048-dimensional in LLAMA-2-7B. Another intuitive baseline we consider is to directly use the validation data $\mathcal{D}_{\text{val}}$ for training, but this drastically degrades performance (Appendix D.3).

5.3. Main results

We present main results of LESS across different models in Table 2, and comparison with baseline approaches in Table 3. We summarize the findings below.

LESS is effective across different models. Table 2 shows that LESS consistently outperforms random selection by 2 to 5 points across all models and evaluation datasets, demonstrating that our data selection approach selects useful data for fine-tuning.

5% of the data often outperforms the full dataset. In Table 2, we compare training with 5% of the data selected by LESS and training with the full dataset. Surprisingly, we discover that in most cases, training with just 5% of

Table 2: Results of LESS (selected with target model’s gradient store) and LESS-T (selected with LLAMA-2-7B’s gradient store) on LLAMA-2-7B, LLAMA-2-13B and MISTRAL-7B. Full denote full dataset, and otherwise we select 5% of the data with random selection (Rand.), LESS-T and LESS. **Bold** numbers denotes the best performing selected subset. Underlined numbers denote that the selected subset outperforms the full dataset. Numbers in the parentheses are standard deviations.

Data percentage	MMLU				TYDIQA				BBH			
	Full (100%)	Rand. (5%)	LESS-T (5%)	LESS (5%)	Full (100%)	Rand. (5%)	LESS-T (5%)	LESS (5%)	Full (100%)	Rand. (5%)	LESS-T (5%)	LESS (5%)
LLAMA-2-7B	51.6	46.5 (0.5)	-	50.2 (0.5)	54.0	52.7 (0.4)	-	56.2 (0.7)	43.2	38.9 (0.5)	-	41.5 (0.6)
LLAMA-2-13B	54.5	53.4 (0.1)	54.6 (0.3)	54.0 (0.7)	54.3	53.0 (1.3)	57.5 (0.8)	<u>54.6</u> (0.3)	50.8	47.0 (1.6)	49.9 (0.5)	50.6 (0.6)
MISTRAL-7B	60.4	60.0 (0.1)	<u>60.6</u> (0.3)	61.8 (0.4)	57.7	56.9 (0.2)	61.7 (1.7)	<u>60.3</u> (2.4)	53.0	54.5 (0.1)	56.0 (0.8)	56.0 (1.0)

Table 3: Comparison of LESS with BM25, DSIR, and RDS methods when training with top 5% of the data on LLAMA-2-7B. Δ denotes the performance gain of LESS over the strongest baseline.

	Rand.	BM25	DSIR	RDS	LESS	Δ
MMLU	46.5 (0.5)	47.6	46.1 (0.3)	45.0 (1.0)	50.2 (0.5)	$\uparrow 2.6$
TYDIQA	52.7 (0.4)	52.7	44.5 (1.7)	46.8 (1.3)	56.2 (0.7)	$\uparrow 3.5$
BBH	38.9 (0.5)	39.8	36.8 (0.1)	36.7 (1.3)	41.5 (0.6)	$\uparrow 1.7$

the data yields better results than using the entire dataset. We hypothesize that the full dataset contains irrelevant or detrimental data points for specific target tasks. This effect is more evident when using stronger base models (e.g., LLAMA-2-13B and MISTRAL-7B).⁸

Data selected using a small model boosts performance for larger and different models. Table 2 evaluates several LESS-T settings. We use LLAMA-2-7B as the selection model $\mathcal{M}_S$ and train either LLAMA-2-13B (i.e., a larger model) and MISTRAL-7B (i.e., a different model family) as the target model $\mathcal{M}_T$. LESS-T outperforms random selection and has a relatively small gap compared to LESS (i.e., directly using LLAMA-2-13B or MISTRAL-7B as the selection model $\mathcal{M}_S$). The strong performance of LESS-T indicates that LESS selects generalizably useful data, so data selection costs can be further amortized across different models. Conceptually, our derivation in Section 2 states that the dataset selected using one model induces strong performance in another one when the inner products of the gradients of different examples are roughly equal for $\mathcal{M}_S$ and $\mathcal{M}_T$. Our experiments support that this finding is true, thereby adding to the growing body of work that small models can effectively select data for other models during pre-training (Xie et al., 2023a; Engstrom et al., 2024) and in-context learning (Wang et al., 2023a).

⁸We note that there may still be useful data in the discarded 95%, and we leave exploring the optimal threshold to future work.

LESS is the only consistently effective approach compared to others. In Table 3, we compare LESS to several baselines. We observe that conventional approaches relying on word frequency statistics (e.g., BM25), n -gram features (e.g., DSIR), or representations (e.g., RDS) show minimal improvement over random selection, and LESS consistently outperforms the strongest baseline. We acknowledge that the baseline methods are less computationally expensive. Nonetheless, our findings suggest that instruction data calls for a refined definition of similarity between data points (i.e., the influence formulation in Definition 3.1). Although the baseline approaches typically select data that is topically or semantically similar to the query and completion, qualitative analysis (§6.2) indicates that LESS is better at selecting data that require the same kind of reasoning process (i.e., how the instruction is executed) as the target task $\mathcal{D}_{\text{val}}$.

Additional models and tasks. We also perform experiments on more model families and tasks. (1) We conduct experiments with Pythia models (Biderman et al., 2023) across different scales: 14M, 410M, 1B, 6.9B, and 12B. Our findings indicate that LESS consistently outperforms random selection across different model scales. Surprisingly, models as small as Pythia-14M can still effectively select data for larger Pythia models despite not being able to solve the task themselves. Please find more details in Appendix D.5. (2) In experiments on GSM8K (Cobbe et al., 2021) and TruthfulQA (Lin et al., 2022), LESS significantly outperforms random selection, and on TruthfulQA, using just 5% of the data even outperforms using the full dataset. Please find more details in Appendix D.6.

6. Analysis

We analyze LESS in two ways. First, we detail the computational cost of LESS and explore how different design choices affect performance (§6.1). Second, we present a qualitative analysis showing that LESS selects data that aligns with the reasoning capabilities needed for the target task (§6.2).

Table 4: Asymptotic complexity, wall-clock runtime (measured as **single A100 GPU hours**) and storage cost associated with each step in LESS. Gradient computation is the most costly step, followed by the warmup LoRA training stage, but this expense is incurred only once. The actual data selection process requires minimal computation.

	Warmup LoRA Training		Gradient Features Computation		Data Selection	
	Complexity	Actual	Complexity	Actual	Complexity	Actual
Compute	$\mathcal{O}(\mathcal{D}_{\text{warmup}} \cdot N)$	6 Hours	$\mathcal{O}(\mathcal{D} \cdot N)$	48 Hours	$\mathcal{O}(\mathcal{D} \cdot \mathcal{D}_{\text{val}} \cdot d)$	< 1 Min
Storage	-	-	$\mathcal{O}(\mathcal{D} \cdot N \cdot d)$	17.7 GB	-	-

Table 5: Performance of using vanilla gradients from off-the-shelf models (e.g., LLAMA-2-7B and LLAMA-2-7B-CHAT), and Adam gradients from LoRA models (trained with a random set of 5%, 25% and 100% of the full dataset for warmup) for data selection. Unk. denotes that the instruction tuning data is unknown.

	LLAMA-2-7B		LoRA Models		
	Base (0%)	Chat (Unk.)	5% (default)	25%	100%
MMLU	46.7	47.9	50.2	51.3	51.6
TYDIQA	52.1	52.2	56.2	57.0	57.9
BBH	39.8	38.6	41.5	41.5	41.9
Avg.	46.2	46.2	49.3	49.9	50.5

6.1. Computational Complexity

Table 4 describes the asymptotic complexity, wall-clock runtime, and the storage cost required for each step of LESS. The reported wall-clock time is measured in **single A100 (80GB) GPU hours**. The gradient feature computation is the most expensive step, and the cost scales linearly with candidate dataset size $|\mathcal{D}|$, number of checkpoints N , and gradient dimension d . Storing the resulting gradient datastore consumes memory that scales linearly with d . Warmup training is also relatively expensive, and the complexity scales with $|\mathcal{D}_{\text{warmup}}|$ and N . However, the expenses incurred during these two stages are one-time costs that can be amortized over many target tasks.

In the following sections, we explore how our choices for $|\mathcal{D}_{\text{warmup}}|$, N , and d impact the performance of LESS. In summary, increasing $\mathcal{D}_{\text{warmup}}$ and N improves the accuracy of the influence estimation (Definition 3.1), but doing so also increases the cost of the warmup training step. Similarly, increasing d improves the utility of the projected gradient features (Johnson & Lindenstrauss, 1984), but doing so also scales the memory and compute required for LESS.

Warmup training is crucial. The warmup training step tunes a model on $\mathcal{D}_{\text{warmup}}$, which is usually a random 5% of the data $\mathcal{D}$. We investigate: (1) if this step is necessary, and (2) if using more data further enhances performance.

Table 6: Number of checkpoints (N) used for select data with LESS. Using fewer checkpoints still outperforms random selection but is less effective.

	MMLU	TYDIQA	BBH	Avg.
Random	46.5 (0.5)	52.7 (0.4)	38.9 (0.5)	46.0
$N = 1$	48.2 (0.4)	54.9 (0.4)	40.2 (0.2)	47.8
$N = 4$ (default)	50.2 (0.5)	56.2 (0.7)	41.5 (0.6)	49.3

For (1), we use pre-trained LLAMA-2-7B and LLAMA-2-7B-CHAT as selection models to create a gradient data store for selecting data.⁹ Table 5 demonstrates that using pre-trained models for data selection significantly hurts the performance of LESS. This observation may stem from the input distribution shift, or more generally, nuances in the training dynamics (see Appendix F). We leave detailed investigations to future work. For (2), we vary the size of $\mathcal{D}_{\text{warmup}}$ to be 5% (the default), 25%, and the entire dataset and compute Inf_{Adam} . Increasing $|\mathcal{D}_{\text{warmup}}|$ improves performance, confirming our hypothesis that estimating Inf_{Adam} (Definition 3.1) more accurately is helpful. Therefore, the warmup phase is crucial, but it does not need to be too long for LESS to yield strong performance.

More checkpoints, better performance. We investigate if using fewer checkpoints from the warmup training stage (i.e., summing over fewer gradient features in Definition 3.1) affects the performance of LESS. Table 6 shows that using only one LoRA checkpoint outperforms random selection but underperforms using four checkpoints. We speculate that this is because the gradient features change substantially at the start of instruction tuning (see Appendix F).

A small projection dimension is sufficient. We study how choosing different gradient projection dimensions d , specifically 1024, 2048, 4096, and 8192 (default), impacts the performance of LESS. Figure 2 shows that all choices of d improve performance over random selection, and as d increases, LESS yields a stronger performance at a higher

⁹Note that we compute Inf_{SGD} instead of Inf_{Adam} , since the pre-training optimization states (i.e., Γ) is not available.

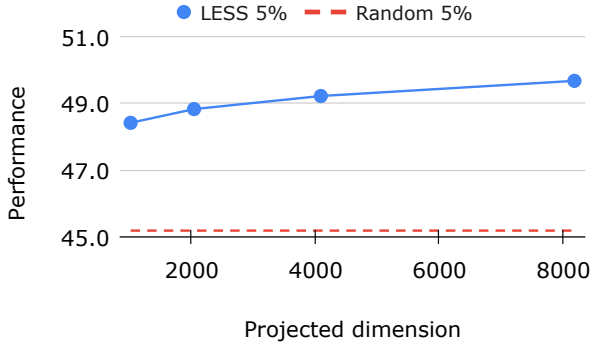


Figure 2: Average performance of LESS on three datasets when using different projected dimension (1024, 2048, 4096 and 8192). We select 5% of the data for instruction tuning. Please refer to Appendix D.4 for task-specific results.

computational cost (see Table 4).

6.2. Qualitative Analysis

We manually compare the data selected by LESS and baseline approaches. Table 16 shows top examples selected by BM25, RDS, and LESS, given a single Bengali question-answering example from the TYDIQA dataset. BM25 and RDS simply match the few-shot data $\mathcal{D}_{\text{val}}$ and select Bengali examples from different tasks (mask recovery and hate speech detection, respectively).¹⁰ In contrast, LESS selects an English example from an open-book question-answering task similar to TYDIQA. This example demonstrates that LESS selects examples with a similar underlying reasoning type without relying heavily on surface textual or linguistic cues. Such examples enable effective zero-shot transfer to the Bengali language, despite being in a different language (Hu et al., 2020). Furthermore, we observe that LESS chooses more examples from the DOLLY dataset for TYDIQA, likely because DOLLY contains many open-book question-answering examples. Our findings suggest that LESS may be used to improve interpretability of large, general-purpose instruction tuning datasets. Appendix K contains more analysis details.

7. Related Work

Curating high-quality instruction tuning data. Using high-quality instruction tuning data can dramatically improve base LLMs. Instruction tuning data is generally of two types: (1) task-based datasets curated from traditional NLP tasks (Wang et al., 2022; Sanh et al., 2022; Wei et al.,

2022b; Longpre et al., 2023), and (2) open-ended instruction following datasets, broadly covering a wide range of topics (Taori et al., 2023; Conover et al., 2023; Köpf et al., 2023; Xu et al., 2023; Mukherjee et al., 2023; Zhou et al., 2023; Ding et al., 2023). Increasing data quality and diversity instead of quantity has been shown to more effectively induce instruction following abilities (Cao et al., 2023; Chen et al., 2023a; Bukharin & Zhao, 2023; Du et al., 2023; Liu et al., 2023; Li et al., 2023). Moreover, (Han, 2023; Lin et al., 2023a) have demonstrated that **in-context learning alone can significantly advance instruction following**. Our work adds to this narrative by selecting high-quality, relevant data from a large pool of available datasets to induce a particular capability in the model.

Coresets and gradient-based data selection. **Data selection has been viewed as a coreset selection problem** (Phillips, 2017), which aims to find a subset of training examples that induces performance similar to training on the full dataset (Toneva et al., 2018; Sener & Savarese, 2018; Coleman et al., 2019; Kaushal et al., 2019; Xia et al., 2020; Mirzasoleiman et al., 2020; Feldman & Zhang, 2020; Killamsetty et al., 2021a;b; Paul et al., 2021; Meding et al., 2021; Mindermann et al., 2022; Sorscher et al., 2022; Xia et al., 2023). These works focus on in-domain coreset selection, whereas **our work involves transfer learning**. Several prior works use pre-defined notions of useful data (Gururangan et al., 2020; Chen et al., 2023b) or n-gram features (Xie et al., 2023b) to select pre-training examples, but **LESS more closely resembles Mirzasoleiman et al. (2020); Wang et al. (2020); Yu et al. (2020b); Killamsetty et al. (2021a)** in its reliance on **gradient information**. Our work is closest to Han et al. (2023), which used model gradients to select pre-training data to improve in-context learning performance. However, our offline approach enables extremely efficient adaptation to new downstream tasks.

Data attribution and influence functions. Our formulation of influence has been used in identifying mislabeled examples (Pruthi et al., 2020), analyzing memorization effects (Feldman & Zhang, 2020), and deriving various interpretability insights (Madsen et al., 2022), but it does not permit counterfactual reasoning. On the other hand, influence functions (Hampel, 1974) can counterfactually reason about model behaviors and training data (Koh & Liang, 2017), but some studies have found limitations (Bae et al., 2022) in its robustness (Basu et al., 2020) and effectiveness (Ilyas et al., 2022). In the LLM setting, influence functions are computationally expensive (Grosse et al., 2023), though concurrent work in Engstrom et al. (2024) proposed using an efficient influence function estimation (Park et al., 2023) to select pre-training data (see Appendix G for a detailed comparison of influence functions and our influence formulation). Our findings suggest that first-order influence

¹⁰The top 10 examples selected by BM25 and RDS are of Bengali translation, Bengali sentiment classification, Bengali mask prediction, etc.

approximations (Pruthi et al., 2020) are effective for data selection in transfer learning settings (Han et al., 2023).

8. Discussion and Future Work

We propose a data selection algorithm LESS based on optimizer-aware influences (§2 and §3). LESS creates a datastore of effective and reusable low-dimensional gradient features to enable efficient data selection (§4). Experiments in §5 demonstrate the effectiveness of LESS compared to relevant baselines and also highlight the ability to use smaller models to select data to train larger models (Table 2). Analysis and ablations in §6 demonstrate that our method selects more interpretable data but can be computationally expensive. We discuss the limitations in detail in Section 8.

In the future, LESS could be **effectively used for test-time adaptation**, allowing the model to adapt on data selected based on the context and demonstrations provided by the users. Additionally, we could **modify the data selection procedure to minimize any differentiable metric** (e.g., a surrogate for **toxicity** and **harmfulness**) instead of just **cross-entropy loss**. Moreover, our experiments and concurrent findings in Engstrom et al. (2024) may prompt further investigation of the utility of gradient features as opposed to surface-form cues in data selection. Insights into the optimization characteristics and sequence-level gradient phenomena (§3.2) are likely of theoretical interest.

Our observation that tuning on more data can degrade performance on a particular task may also be intimately connected to the catastrophic forgetting behaviors observed in LLMs (Lin et al., 2023b).

Limitations

We discuss the limitations of our approach here.

1. **Warmup training phase is required.** The warmup training phase is crucial for obtaining useful gradient features for data selection. It is evident that we must fine-tune the base model to align with the specific data distribution we intend to select from, because directly utilizing gradients from pre-trained models like LLAMA-2-7B or LLAMA-2-7B-CHAT is not as effective (Table 5). This additional step introduces increased complexity and computational load to LESS.
2. **Average gradients over completion tokens.** The average gradient of all the completion tokens is used as a surrogate to represent the entire sequence. In scenarios involving longer sequences in training or validation data, such as open-ended generation, this aggregation could become more ambiguous and less effective. In Section 3.2, we highlight how this operation can cause anomalies in the data selection pipeline.
3. **Minimizing loss does not monotonically increase performance.** Unlike standard vision settings, minimizing the validation loss (i.e., cross entropy loss) does not always improve the task performance in instruction tuning. This is because instruction tuning tasks can generally require the model to produce long-form generations. Although perplexity aligns well with task performance in base LLMs (Xia et al.), but the relationship is much more obscure in fine-tuning or instruction tuning (Tay et al., 2021; Zhou et al., 2023). LESS is motivated by selecting datapoints that minimize the validation loss (Section 2) and this strategy often induces models with high accuracy (Table 2), but minimizing loss does not monotonically improve accuracy (see our discussion in Appendix I).
4. **Linearity in data selection.** The first-order approximation in Section 2 ignores the effect of adding multiple datapoints together. In particular, two duplicate points would be scored equally highly and thought to doubly improve the model, though this is likely not the case. Preliminary theory in Saunshi et al. (2023) discusses when influences may add linearly. However, several more sophisticated algorithms have been proposed to more flexibly model interactions across datapoints at a significantly higher computational cost (Søgaard et al., 2021; Guu et al., 2023). Unfortunately, such algorithms are too expensive in the language model setting.

Acknowledgments

We thank Logan Engstrom, Xiaochuang Han, Kaifeng Lyu, Abhishek Panigrahi, Yizhong Wang, Zhaofeng Wu, Dingli Yu, and Haoyu Zhao for their insightful discussions on the project. We thank Tianyu Gao, Jiayi Geng, Zirui Wang, Howard Yen, and other members of Princeton NLP for proofreading this work. MX is supported by the Bloomberg Data Science Ph.D. Fellowship. SM and SA are supported by NSF and ONR. This research is also funded by the National Science Foundation (IIS-2211779).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Anthropic. Introducing claude, 2023. URL <https://www.anthropic.com/index/introducing-claude>.
- Arora, S., Li, Y., Liang, Y., Ma, T., and Risteski, A. A latent

- variable model approach to pmi-based word embeddings. *Transactions of the Association for Computational Linguistics*, 4:385–399, 2016.
- Arora, S., Du, S. S., Hu, W., Li, Z., Salakhutdinov, R. R., and Wang, R. On exact computation with an infinitely wide neural net. *Advances in neural information processing systems*, 32, 2019.
- Bae, J., Ng, N. H., Lo, A., Ghassemi, M., and Grosse, R. B. If influence functions are the answer, then what is the question? In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=hzbguA9zMJ>.
- Basu, S., Pope, P., and Feizi, S. Influence functions in deep learning are fragile. In *International Conference on Learning Representations*, 2020.
- Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O’Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.
- Bukharin, A. and Zhao, T. Data diversity matters for robust instruction tuning. *arXiv preprint arXiv:2311.14736*, 2023.
- Cao, Y., Kang, Y., and Sun, L. Instruction mining: High-quality instruction data selection for large language models. *arXiv preprint arXiv:2307.06290*, 2023.
- Chen, L., Li, S., Yan, J., Wang, H., Gunaratna, K., Yadav, V., Tang, Z., Srinivasan, V., Zhou, T., Huang, H., et al. AlpagaSus: Training a better alpaca with fewer data. *arXiv preprint arXiv:2307.08701*, 2023a.
- Chen, M. F., Roberts, N., Bhatia, K., WANG, J., Zhang, C., Sala, F., and Re, C. Skill-it! a data-driven skills framework for understanding and training language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023b. URL <https://openreview.net/forum?id=IoizwO1NLf>.
- Clark, J. H., Choi, E., Collins, M., Garrette, D., Kwiatkowski, T., Nikolaev, V., and Palomaki, J. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 2020.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Coleman, C., Yeh, C., Mussmann, S., Mirzasoleiman, B., Bailis, P., Liang, P., Leskovec, J., and Zaharia, M. Selection via proxy: Efficient data selection for deep learning. In *International Conference on Learning Representations*, 2019.
- Conover, M., Hayes, M., Mathur, A., Xie, J., Wan, J., Shah, S., Ghodsi, A., Wendell, P., Zaharia, M., and Xin, R. Free Dolly: Introducing the world’s first truly open instruction-tuned LLM, 2023.
- Ding, N., Chen, Y., Xu, B., Qin, Y., Zheng, Z., Hu, S., Liu, Z., Sun, M., and Zhou, B. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.
- Du, Q., Zong, C., and Zhang, J. Mods: Model-oriented data selection for instruction tuning. *arXiv preprint arXiv:2311.15653*, 2023.
- Engstrom, L., Feldmann, A., and Madry, A. Dsdm: Model-aware dataset selection with datamodels, 2024.
- Feldman, V. and Zhang, C. What neural networks memorize and why: Discovering the long tail via influence estimation. In *Advances in Neural Information Processing Systems*, volume 33, pp. 2881–2891, 2020.
- Google. An important next step on our ai journey, 2023. URL <https://blog.google/technology/ai/bard-google-ai-search-updates/>.
- Grosse, R., Bae, J., Anil, C., Elhage, N., Tamkin, A., Tajdini, A., Steiner, B., Li, D., Durmus, E., Perez, E., Hubinger, E., Lukošiušė, K., Nguyen, K., Joseph, N., McCandlish, S., Kaplan, J., and Bowman, S. R. Studying large language model generalization with influence functions, 2023.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., and Smith, N. A. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020.
- Guu, K., Webson, A., Pavlick, E., Dixon, L., Tenney, I., and Bolukbasi, T. Simfluence: Modeling the influence of individual training examples by simulating training runs, 2023.
- Hampel, F. R. The influence curve and its role in robust estimation. *Journal of the american statistical association*, 69(346):383–393, 1974.
- Han, X. In-context alignment: Chat with vanilla language models before fine-tuning. *arXiv preprint arXiv:2308.04275*, 2023.

- Han, X., Simig, D., Mihaylov, T., Tsvetkov, Y., Celikyilmaz, A., and Wang, T. Understanding in-context learning via supportive pretraining data. In *The 61st Annual Meeting Of The Association For Computational Linguistics*, 2023.
- Hanawa, K., Yokoi, S., Hara, S., and Inui, K. Evaluation of similarity-based explanations. In *International Conference on Learning Representations*, 2020.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2020.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Hu, J., Ruder, S., Siddhant, A., Neubig, G., Firat, O., and Johnson, M. XTREME: A massively multilingual multi-task benchmark for evaluating cross-lingual generalisation. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 4411–4421. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/hu20b.html>.
- Ilyas, A., Park, S. M., Engstrom, L., Leclerc, G., and Madry, A. Datamodels: Predicting predictions from training data. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.
- Jacot, A., Gabriel, F., and Hongler, C. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Johnson, W. B. and Lindenstrauss, J. Extensions of lipschitz mappings into hilbert space. *Contemporary mathematics*, 26:189–206, 1984.
- Kaushal, V., Iyer, R., Kothawade, S., Mahadev, R., Doctor, K., and Ramakrishnan, G. Learning from less data: A unified data subset selection and active learning framework for computer vision. In *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 1289–1299. IEEE, 2019.
- Killamsetty, K., Durga, S., Ramakrishnan, G., De, A., and Iyer, R. Grad-match: Gradient matching based data subset selection for efficient deep model training. In *International Conference on Machine Learning*, pp. 5464–5474. PMLR, 2021a.
- Killamsetty, K., Zhao, X., Chen, F., and Iyer, R. Retrieve: Coreset selection for efficient and robust semi-supervised learning. In *Advances in Neural Information Processing Systems*, volume 34, pp. 14488–14501, 2021b.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In Bengio, Y. and LeCun, Y. (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- Koh, P. W. and Liang, P. Understanding black-box predictions via influence functions. In *International conference on machine learning*, pp. 1885–1894. PMLR, 2017.
- Köpf, A., Kilcher, Y., von Rütte, D., Anagnostidis, S., Tam, Z.-R., Stevens, K., Barhoum, A., Duc, N. M., Stanley, O., Nagyfi, R., et al. OpenAssistant conversations—democratizing large language model alignment. In *Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)*, 2023.
- Li, Y., Hui, B., Xia, X., Yang, J., Yang, M., Zhang, L., Si, S., Liu, J., Liu, T., Huang, F., et al. One shot learning as instruction data prospector for large language models. *arXiv preprint arXiv:2312.10302*, 2023.
- Lin, B. Y., Ravichander, A., Lu, X., Dziri, N., Sclar, M., Chandu, K., Bhagavatula, C., and Choi, Y. The unlocking spell on base LLMs: Rethinking alignment via in-context learning. *arXiv preprint arXiv:2312.01552*, 2023a.
- Lin, S., Hilton, J., and Evans, O. TruthfulQA: Measuring how models mimic human falsehoods. In *Association for Computational Linguistics (ACL)*, 2022.
- Lin, Y., Tan, L., Lin, H., Zheng, Z., Pi, R., Zhang, J., Diao, S., Wang, H., Zhao, H., Yao, Y., et al. Speciality vs generality: An empirical study on catastrophic forgetting in fine-tuning foundation models. *arXiv preprint arXiv:2309.06256*, 2023b.
- Liu, W., Zeng, W., He, K., Jiang, Y., and He, J. What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. *arXiv preprint arXiv:2312.15685*, 2023.
- Longpre, S., Hou, L., Vu, T., Webson, A., Chung, H. W., Tay, Y., Zhou, D., Le, Q. V., Zoph, B., Wei, J., et al. The flan collection: Designing data and methods for effective instruction tuning. *arXiv preprint arXiv:2301.13688*, 2023.
- Madsen, A., Reddy, S., and Chandar, S. Post-hoc interpretability for neural nlp: A survey. *ACM Computing Surveys*, 55(8):1–42, 2022.

- Malladi, S., Lyu, K., Panigrahi, A., and Arora, S. On the scales and scaling rules for adaptive gradient algorithms. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 7697–7711. Curran Associates, Inc., 2022.
- Malladi, S., Wettig, A., Yu, D., Chen, D., and Arora, S. A kernel-based view of language model fine-tuning. In *International Conference on Machine Learning*, pp. 23610–23641. PMLR, 2023.
- Meding, K., Buschhoff, L. M. S., Geirhos, R., and Wichmann, F. A. Trivial or impossible—dichotomous data difficulty masks model differences (on imagenet and beyond). In *International Conference on Learning Representations*, 2021.
- Mindermann, S., Brauner, J. M., Razzak, M. T., Sharma, M., Kirsch, A., Xu, W., Höltgen, B., Gomez, A. N., Morisot, A., Farquhar, S., et al. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning*, pp. 15630–15649. PMLR, 2022.
- Mirzasoleiman, B., Bilmes, J., and Leskovec, J. Coresets for data-efficient training of machine learning models. In *International Conference on Machine Learning*, pp. 6950–6960. PMLR, 2020.
- Mukherjee, S., Mitra, A., Jawahar, G., Agarwal, S., Palangi, H., and Awadallah, A. Orca: Progressive learning from complex explanation traces of gpt-4. *arXiv preprint arXiv:2306.02707*, 2023.
- Nguyen, T., Chen, Z., and Lee, J. Dataset meta-learning from kernel ridge-regression. In *International Conference on Learning Representations*, 2021.
- OpenAI. OpenAI: Introducing ChatGPT, 2022. URL <https://openai.com/blog/chatgpt>.
- OpenAI. OpenAI: GPT-4, 2023. URL <https://openai.com/research/gpt-4>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- Park, S. M., Georgiev, K., Ilyas, A., Leclerc, G., and Madry, A. Trak: Attributing model behavior at scale. In *International Conference on Machine Learning (ICML)*, 2023.
- Paul, M., Ganguli, S., and Dziugaite, G. K. Deep learning on a data diet: Finding important examples early in training. In *Advances in Neural Information Processing Systems*, volume 34, pp. 20596–20607, 2021.
- Phillips, J. M. Coresets and sketches. In *Handbook of discrete and computational geometry*, pp. 1269–1288. Chapman and Hall/CRC, 2017.
- Pruthi, G., Liu, F., Kale, S., and Sundararajan, M. Estimating training data influence by tracing gradient descent. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 19920–19930. Curran Associates, Inc., 2020.
- Robertson, S., Zaragoza, H., et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.
- Sanh, V., Webson, A., Raffel, C., Bach, S. H., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Le Scao, T., Raja, A., et al. Multitask prompted training enables zero-shot task generalization. In *ICLR 2022-Tenth International Conference on Learning Representations*, 2022.
- Saunshi, N., Gupta, A., Braverman, M., and Arora, S. Understanding influence functions and datamodels via harmonic analysis. In *The Eleventh International Conference on Learning Representations*, 2023.
- Sener, O. and Savarese, S. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations*, 2018.
- Søgaard, A. et al. Revisiting methods for finding influential examples. *arXiv preprint arXiv:2111.04683*, 2021.
- Sorscher, B., Geirhos, R., Shekhar, S., Ganguli, S., and Morcos, A. Beyond neural scaling laws: beating power law scaling via data pruning. *Advances in Neural Information Processing Systems*, 35:19523–19536, 2022.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q., Chi, E., Zhou, D., et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Tay, Y., Dehghani, M., Rao, J., Fedus, W., Abnar, S., Chung, H. W., Narang, S., Yogatama, D., Vaswani, A., and Metzler, D. Scale efficiently: Insights from pretraining and finetuning transformers. In *International Conference on Learning Representations*, 2021.

- Toneva, M., Sordoni, A., des Combes, R. T., Trischler, A., Bengio, Y., and Gordon, G. J. An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations*, 2018.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and finetuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Wang, X., Pham, H., Michel, P., Anastasopoulos, A., Carbonell, J., and Neubig, G. Optimizing data usage via differentiable rewards. In *International Conference on Machine Learning*, pp. 9983–9995. PMLR, 2020.
- Wang, X., Zhu, W., Saxon, M., Steyvers, M., and Wang, W. Y. Large language models are latent variable models: Explaining and finding good demonstrations for in-context learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023a.
- Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., and Hajishirzi, H. Self-Instruct: Aligning language models with self-generated instructions. In *Association for Computational Linguistics (ACL)*, pp. 13484–13508.
- Wang, Y., Mishra, S., Alipoormolabashi, P., Kordi, Y., Mirzaei, A., Naik, A., Ashok, A., Dhanasekaran, A. S., Arunkumar, A., Stap, D., et al. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 5085–5109, 2022.
- Wang, Y., Ivison, H., Dasigi, P., Hessel, J., Khot, T., Chandu, K. R., Wadden, D., MacMillan, K., Smith, N. A., Beltagy, I., et al. How far can camels go? exploring the state of instruction tuning on open resources. In *Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)*, 2023b.
- Wei, A., Hu, W., and Steinhardt, J. More than a toy: Random matrix models predict how real-world neural representations generalize. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 23549–23588. PMLR, 17–23 Jul 2022a. URL <https://proceedings.mlr.press/v162/wei22a.html>.
- Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022b.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35: 24824–24837, 2022c.
- Xia, M., Artetxe, M., Zhou, C., Lin, X. V., Pasunuru, R., Chen, D., Zettlemoyer, L., and Stoyanov, V. Training trajectories of language models across scales. In *Association for Computational Linguistics (ACL)*, pp. 13711–13738.
- Xia, M., Anastasopoulos, A., Xu, R., Yang, Y., and Neubig, G. Predicting performance for natural language processing tasks. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8625–8646, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.764. URL <https://aclanthology.org/2020.acl-main.764>.
- Xia, X., Liu, J., Yu, J., Shen, X., Han, B., and Liu, T. Moderate coreset: A universal method of data selection for real-world data-efficient deep learning. In *The Eleventh International Conference on Learning Representations*, 2023.
- Xie, S. M., Pham, H., Dong, X., Du, N., Liu, H., Lu, Y., Liang, P., Le, Q. V., Ma, T., and Yu, A. W. DoReMi: Optimizing data mixtures speeds up language model pre-training. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023a. URL <https://openreview.net/forum?id=1XuByUeHhd>.
- Xie, S. M., Santurkar, S., Ma, T., and Liang, P. Data selection for language models via importance resampling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023b.
- Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., and Jiang, D. WizardLM: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.
- Yang, G. and Hu, E. J. Tensor programs iv: Feature learning in infinite-width neural networks. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 11727–11737. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/yang21c.html>.
- Yu, D., Wang, R., Li, Z., Hu, W., Salakhutdinov, R., Arora, S., and Du, S. S. Enhanced convolutional neural tangent kernels, 2020a. URL <https://openreview.net/forum?id=BkgNqkHFPr>.

Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33: 5824–5836, 2020b.

Zhang, R., Isola, P., Efros, A. A., Shechtman, E., and Wang, O. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 586–595, 2018.

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., et al. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A. Training

A.1. Training Datasets

For training, we utilize the same four processed datasets as referenced in (Wang et al., 2023b). Each of these datasets is either annotated or authored by humans. Further details about these datasets can be found in Table 7. The FLAN V2 and CoT datasets are derived from existing NLP datasets, while DOLLY and OPEN ASSISTANT 1 consist of open-ended generation examples featuring answers written by humans. These datasets vary significantly in format, sequence length, and underlying tasks, reflecting the heterogeneity of instruction-tuning datasets. We adopt the following 'Tulu' format from the study by Wang et al. (2023b) to standardize the format of these datasets.

```
<|user|>
Alice's parents have three daughters: Amy, Jessy, and what's the name of the third daughter?

<|assistant|>
The name of the third daughter is Alice
```

Table 7: Details of training dataset from Wang et al. (2023b). Len. is short for token length.

Dataset	# Instance	Sourced from	# Rounds	Prompt Len.	Completion Len.
FLAN V2	100,000	NLP datasets and human-written instructions	1	355.7	31.2
CoT	100,000	NLP datasets and human-written CoTs	1	266	53.2
DOLLY	15,011	Human-written from scratch	1	118.1	91.3
OPEN ASSISTANT 1	55,668	Human-written from scratch	1.6	34.8	212.5

A.2. Training Details

All experiments were conducted using the parameter-efficient fine-tuning method LoRA (Hu et al., 2021). We employed a learning rate scheduler with linear warm-up and cosine decay, reaching a peak learning rate of 2×10^{-5} . A batch size of 128 was used, and training was carried out for 4 epochs across all selected datasets. We initially conducted experiments to align the number of training steps between small and large datasets. However, increasing the number of epochs on small datasets did not yield performance improvements. Consequently, we opted for a fixed number of epochs in all experiments.

For the LoRA module, we specified a rank of 128, an α value of 512, a dropout rate of 0.1, and learned LoRA matrices for all attention matrices. This configuration resulted in 135 million trainable parameters for LLAMA-2-7B, constituting 1.95% of the original LLAMA-2-7B model. For MISTRAL-7B, there were 109 million trainable parameters, accounting for 1.48% of the original MISTRAL-7B model. Lastly, the LLAMA-2-13B model comprised 209 million trainable parameters, representing 1.59% of the original LLAMA-2-13B model.

In each experiment, we performed three trials using distinct random seeds. When employing random selection approaches, this entailed selecting three different random subsets from the training dataset. In the case of our approach (LESS), it involved conducting warmup training with various subsets of the training data and subsequently selecting different subsets for each trial from each warmup-trained model. We used consistent optimization seeds across all experiments.

B. Evaluation Details

We follow Wang et al. (2023b) to evaluate the performance of the models on the target tasks. For MMLU, we measure the 5-shot accuracy of the test set averaged across 57 subtasks. For TYDIQA, we measure the 1-shot macro-averaged F1 score across all 11 languages. We adopt the gold-passage setup where one passage containing the reference answer is provided to the model. For BBH, we report the average 3-shot exact match score across all tasks. Chain-of-thought reasoning is provided in each in-context learning example to prompt the model to generate chain-of-thought reasoning traces for test examples. We evaluate on the validation set $\mathcal{D}_{\text{val}}$ (the same reference set used for data selection) at the end of each epoch and select the best checkpoint to evaluate on the final test set for each experiment. Note that this procedure might introduce some bias to the final test set, given that the validation set is relatively small (e.g., TYDIQA only has 9 validation examples in total). However, in practice, we don't find this bias to affect the comparisons between different methods.

C. Baseline Details

For baseline approaches that involve stochasticity (i.e., all except BM25), we perform three runs with different random seeds and report the average performance and standard deviation.

C.1. BM25

We implement the BM25 baseline with the `rank_bm25`¹¹ package. We employ the same approach to aggregating scores across subtasks as Equation (2), except using the BM25 scores instead of the gradient similarity scores.

C.2. DSIR

DSIR (Xie et al., 2023b) offers a lightweight approach to select data based on specific target distributions using n-gram features. However, we find it unsuitable for our specific setting. The primary issue stems from the difficulty in learning a reliable n-gram model with the limited data available in our target distribution, leading to a sparse n-gram table. We hypothesize that this sparsity arises from the trade-off between bucket size and estimation accuracy for the source and target distribution: larger buckets result in a highly sparse table with unreliable estimations for data selection, while smaller buckets lead to more hash collisions for the data in the source distribution. In our empirical observations, we noticed that DSIR tends to prefer shorter sequences, which can harm performance (see Table 13 for similar evidence). The tendency of DSIR to select short sequences is also mentioned in Appendix J of Xie et al. (2023b), where the authors impose heuristic quality filters on top of the DSIR-selected datasets.

D. More Experiment Results

D.1. Comparing SGD and Adam Optimizers for Instruction Tuning

The Adam optimizer (Kingma & Ba, 2015) is commonly employed for fine-tuning large language models. However, SGD is a more desirable optimizer due to its lower memory usage; moreover, the influence formulation we introduce in §2 applies only to SGD. Here, we verify that using SGD to instruction tune models results in markedly worse performance Table 8. Our findings consistently demonstrate that using the Adam optimizer yields better results across all three datasets. This validates the widespread practice of utilizing the Adam optimizer for fine-tuning large language models.

Table 8: Comparing using the SGD and Adam optimizers for instruction tuning. We use a random subset of 25% data for comparison. We find that using the Adam optimizer consistently outperforms SGD on all three datasets.

	MMLU	TYDIQA	BBH	Average
SGD	45.5 (0.3)	47.3 (1.2)	39.2 (0.5)	44.0
Adam	47.2 (0.8)	53.1 (1.3)	40.4 (1.5)	46.9

D.2. Selecting Data for SGD, SignGD, and Adam Gradients

In order to compute the influence when training with the Adam optimizer (Definition 3.1), we need to access the Adam optimizer update Γ . Γ relies on the moving moment estimates that Adam tracks. In the early stages of training, Malladi et al. (2022) proved that the parameter updates induced by Adam can be approximated well by the sign of the gradient (this optimizer is known as SignGD). Intuitively, this is because the moment estimates m and v are close to their initialization and thus essentially perform coordinate-wise normalization on the gradient. Later empirical and theoretical in Malladi et al. (2023) suggested that fine-tuning language models occurs in the early stages of Adam, so signGD may be a decent approximation of fine-tuning LLMs with the Adam optimizer.

In the current paper, we obtain Γ through the optimizer states computed over the course of a short warmup training period. Here, we ablate our stated approach against approximating Γ as $\text{sign}(\nabla \ell(\cdot; \theta))$. One may also think this SignGD formulation could circumvent the gradient norm problems reported in §3.2.

One of our key contributions is to extend the influence formulation in §2 to operate with the Adam optimizer. However, due

¹¹https://github.com/dorianbrown/rank_bm25

to the approximations described above, the influences we compute in Definition 3.1 may not be as accurate as we hope. In that case, we may be better off using the more precise influences provided by the SGD formulation in Equation (1).

We empirically compare the efficacy of these three influence formulations (SGD, SignGD, and Adam) for data selection and present the results in Table 9. In this setup, we perform warmup training using all the available training data ($\mathcal{D}_{\text{warmup}} = \mathcal{D}$) and select the top 5% of the data for instruction tuning. In summary, we find that using Adam influence formulation generally matches or outperforms the SGD and SignGD formulations, demonstrating the utility of our optimizer-aware influence formulation in Definition 3.1.

Table 9: Comparing the data selection capabilities of the SGD, SignGD, and Adam (Definition 3.1) formulations of influence. We find that using the Adam formulation generally matches or outperforms using the SGD and SignGD ones, showing the utility of our optimizer-aware influence formulation.

	Random	SGD	SignGD	Adam
MMLU	46.5	50.9	49.7	51.6
TYDIQA	52.7	58.1	54.8	57.9
BBH	38.9	40.0	39.0	41.9
Avg.	46.0	49.7	47.8	50.5

D.3. Comparison to Base Models, and Training with Validation Data

Table 10: Evaluation results with the base model (without instruction tuning), and with instruction tuning on the validation set. Finetuning on the validation set alone rarely improves the performance on the test set.

	LLAMA-2-7B				LLAMA-2-13B				MISTRAL-7B			
	Base	Valid.	Rand. 5%	LESS 5%	Base	Valid.	Rand. 5%	LESS 5%	Base	Valid.	Rand 5%	LESS 5%
MMLU	45.6	47.3 (0.3)	46.5 (0.5)	50.2 (0.5)	55.3	54.8 (0.4)	53.4 (0.1)	54.0 (0.7)	62.4	58.8 (0.4)	60.0 (0.1)	61.8 (0.4)
TYDIQA	46.4	52.8 (2.5)	52.7 (0.4)	56.2 (0.7)	53.1	50.8 (1.4)	53.0 (1.3)	54.6 (0.3)	49.6	45.4 (4.3)	56.9 (0.2)	60.3 (2.4)
BBH	38.3	36.5 (0.9)	38.9 (0.5)	41.5 (0.6)	46.2	45.8 (0.9)	47.0 (1.6)	50.6 (0.6)	54.9	55.5 (0.7)	54.5 (0.1)	56.8 (1.0)

Instruction tuning could degrade performance. In Table 10, we compare results from instruction tuning to the results directly from the base model without instruction tuning. Although instruction tuning generally enhances performance, there are instances where it may result in a decline. A notable example is observed in the MMLU dataset, where instruction tuning using the 5% data subset chosen by LESS yields slightly inferior results compared to the base model. This suggests that the chosen data may not significantly improve the performance of the target task for some models, especially more capable ones such as LLAMA-2-13B and MISTRAL-7B. Additionally, it could imply that the pool of training data we’re selecting from lacks further beneficial examples for the specified task. Experimenting with a larger pool of instruction tuning could possibly benefit the target task more. Despite this, it is important to note that LESS remains competitive to other approaches. By scaling up the compute to have a better estimate of the influence, we expect that the performance of LESS will further improve, as indicated in Section 6.1.

Training with validation data rarely helps. An additional straightforward baseline involves using the few-shot validation data for training. Specifically, we train the model for 100 steps with the standard hyperparameters, and evaluate the model on the validation set for every 10 steps, and report the test score of the best checkpoint. The results of this approach are shown in Table 10. Here, we compare it with the performance of the standard LLAMA-2-7B model and the outcomes when fine-tuning the model with either 5% of data chosen at random or 5% selected by LESS. We observe that training solely with in-domain validation data rarely provides a benefit over not training the model at all, and it even surpasses the performance achieved by randomly selecting 5%(13,000) data points. Nevertheless, LESS demonstrates a superior ability to identify data points more relevant to the target task, significantly outperforming the strategy of training with only validation data.

D.4. Detailed Results on Using Different Projection Dimensions

Recall that the Johnson-Lindenstrauss lemma (Johnson & Lindenstrauss, 1984) states that random projections preserve inner products (e.g., the influence formulation in Definition 3.1) as long as the projected dimension d is large enough. Here, we

ablate against different projection dimensions to find what empirical choice of d is “large enough”. In this setup, we perform warmup training using all the available training data ($\mathcal{D}_{\text{warmup}} = \mathcal{D}$) and select the top 5% of the data for instruction tuning. We present the detailed results of projecting the LoRA gradients to different dimensions including 1024, 2048, 4096 and 8192 in Table 11. We find that even using a small dimension still outperforms the random selection performance. Using a larger dimension for projected gradients is better. This signifies that there is still potential to improve the performance by using a larger projection dimension, but one must trade off this consideration with the increased computational and memory cost. Note that due to computational constraint, we only run experiment once with one fixed seed.

Table 11: Performance of using different projected gradient dimensions for data selection. We find that even using a small dimension still outperforms the random selection performance. Using a larger dimension for projected gradients is better.

	Random	Projected Gradient Dimension			
		1024	2048	4096	8192
MMLU	46.5	50.7	51.2	50.5	51.1
TydiQA	52.7	55.3	56.3	56.8	56.6
BBH	38.9	39.3	39.0	40.4	41.3
Average	45.2	48.4	48.8	49.2	49.7

D.5. Results on Pythia Models

To explore how the data selection model in LESS scales with increasing model capabilities, we perform experiments using LESS with a series of Pythia models (Biderman et al., 2023) of sizes 14M, 410M, and 1B to select data for training models of 14M, 410M, 1B, 6.9B, and 12B. Due to constraints on time and computational resources, we conduct these experiments to select data for the TydiQA benchmark using only one seed and present the results in Table 12.

	base (w/o FT)	Random	Data selection with LESS by			
			Pythia-14m	Pythia-410m	Pythia-1b	Full
Amount of FT data	0%	5%	5%	5%	5%	100%
Pythia-14m	0.0	0.0	0.0	0.1	0.0	0.0
Pythia-410m	3.8	7.8	8.7	9.4	8.8	8.6
Pythia-1b	9.2	12.3	12.9	11.5	17.2	13.7
Pythia-6.9b	17.1	27.2	27.2	28.0	30.9	35.5
Pythia-12b	17.4	14.9	17.7	19.4	32.6	33.2
Average	9.5	12.4	13.3	13.7	17.9	18.2
LLAMA-2-7B	46.4	52.7	48.7	53.0	51.7	54.0

Table 12: Performance comparison of using different Pythia models for data selection.

Firstly, LESS consistently performs well with Pythia models, almost always surpassing the baseline of selecting a random 5% of the data. Furthermore, the approach of using one model to select data for training another (i.e., LESS-T) proves effective within the Pythia model family. Interestingly, small models such as the 14M can be effective data selectors despite being unable to solve TydiQA by themselves. This suggests that LESS can be made highly efficient using very small models for the gradient datastore, as they can effectively select data for training larger models. However, larger models demonstrate even greater effectiveness as data selectors, as illustrated in the last row of the table. Lastly, the data selection by Pythia models does not translate effectively to Llama models. Data selected by Pythia models fails to outperform random selection when used to train Llama2 models. Conversely, data chosen by Llama2 models effectively trains Mistral models. This indicates that the success of data transfer between models can be unpredictable, particularly when the capabilities of the models differ significantly like Pythia and Llama2.

D.6. Results on GSM8K and TruthfulQA

We follow reviewers’ suggestions to run experiments on datasets GSM8K (Cobbe et al., 2021) and TruthfulQA (Lin et al., 2022) that are more considered as out of domain with regard to the training data. We evaluate GSM8K using the 8-shot setting with chain of thought, following the methodology of Wang et al. (2023b). Similarly, we use random 10 examples from each dataset as the validation set. We present the results in Appendix D.6. In summary, we find that LESS consistently outperforms random selection by a large margin. However, selecting only 5% of the data to solve GSM8K does not outperform using the entire dataset. This is because the COT portion of our training data contains many in-domain examples for GSM8K, so it is likely not a good idea to use only a small subset of the available data to train the model for that task. LESS’ utility in selecting data for TruthfulQA indicates its potential to make models not only more capable but also more faithful and truthful.

	GSM8K-COT (Exact Match)	TruthfulQA (MC2)
w/o Finetuning (0%)	14.0	39.2
Random (5%)	17.0	41.0
LESS (5%)	21.0	48.0
Full (100%)	30.5	44.0

E. Nuances in Algorithm Design

In Sections 2 and 3, we outline how first-order approximations of influence functions can be used to design optimizer-aware data selection algorithms. Here, we discuss some of the nuances in designing such an algorithm for the practical hyperparameter settings used in instruction tuning.

Batch Size. The per-step influence and trajectory-wise influence of a training datapoint z on a validation datapoint z' is characterized under the assumption that the batch size is 1. Standard training setups use a larger batch size, but it is difficult to characterize how training on a batch of examples can change $\ell(z'; \theta)$. However, under the assumption that the first-order approximation is sufficiently powerful to describe training dynamics, we can directly use the linearity of the gradient to derive the influence of a single datapoint within a batch. In particular, when training with SGD on a batch B_t of size B , we can write:

$$\begin{aligned} \ell(z'; \theta^{t+1}) - \ell(z'; \theta^t) &\approx -\eta_t \langle \nabla \ell(z'; \theta^t), \nabla \ell(B_t; \theta^t) \rangle \\ &= -\frac{\eta_t}{B} \sum_{x \in B_t} \langle \nabla \ell(z'; \theta^t), \nabla \ell(x; \theta^t) \rangle \end{aligned}$$

An analogous formula follows straightforwardly for Adam. The $1/B$ factor is shared across all datapoints and thus does not affect the ranking of the matching scores. Therefore, selecting examples as if the batch size were 1 is the same as selecting examples for a larger batch size. Intuitively, this is because the interaction between examples within a batch must be negligible in order for the first-order approximation to be accurate. We remark that the equal division of the influence across constituents of a mini-batch cannot circumvent the gradient norm problem presented in §3.2. We discuss this point in more detail in Appendix H.

Learning Rate Schedule. Instruction tuning generally uses a linearly decaying learning rate schedule, which means that each example (or batch) within an epoch has a different learning rate depending on the time step that it was sampled. The formulation in Definition 3.1 may then select a datapoint solely because it would be sampled earlier in the training run (i.e., with a larger η_t). We want our method to yield an interpretable dataset, so we instead use the average epoch-wise learning rate as η_t .

F. Comparison to “Kernel Behavior” (Malladi et al., 2023; Jacot et al., 2018).

The design of LESS relies on the assumption that the first-order approximation of loss dynamics is sufficiently accurate (see Section 2). A related hypothesis is the idea that the instruction tuning optimization procedure exhibits “kernel

behavior”.¹² Here, we highlight the nuanced difference between kernel behavior and the current assumption, emphasizing why kernel behavior is difficult to measure in the instruction tuning setting.

Formal Difference Kernel behavior describes the gradient of the output of the model (i.e., the logits) with respect to the model parameters. In contrast, the first-order approximation in Section 2 is for the gradient of the *loss* with respect to the parameters. These two quantities are related by the derivative of the loss with respect to the output of the model. Thus, conceptually, kernel behavior and the first-order approximation of the loss dynamics are similar, but the two phenomena correspond to different formal descriptions of model training. Theory in Yang & Hu (2021); Malladi et al. (2023) indicates that there is some subtlety in whether one phenomenon implies the other holds. In particular, most kernel-based analyses minimize the mean squared error loss instead of the cross-entropy loss, the latter of which introduces technical complications in the analysis but is the standard choice for instruction tuning.

F.1. Does instruction tuning exhibit kernel behavior?

Malladi et al. (2023) suggested that fine-tuning on GLUE-style classification tasks can sometimes exhibit kernel behavior because the pre-trained model is already fairly strong at the downstream task. This assumption crucially relied on appending a very good prompt to the downstream task input, so that labeling the input amounted to predicting the next word. With such a prompt, fine-tuning on the downstream task amounted to the same as continued pre-training. One could argue that the instruction in instruction tuning provides a prompt for the completion, but the completion can be much longer than the single-word labels used in classification tasks.

Gradient features change substantially at the start of instruction tuning. We perform ablations comparing gradient features collected from the pre-trained model and features collected over the course of the warmup training period (Table 6). We find that it is important to incorporate gradients taken over the course of the warmup training, which suggests that the gradient features change substantially at the start of instruction tuning. On the other hand, results in Table 5 demonstrate that lengthening the warmup period yields marginal gains in performance, suggesting that the gradient features don’t change much in the intermediate and late phases of instruction tuning. We emphasize that especially in the case of the cross entropy objective, it is possible for the gradient features in Definition 3.1 to change substantially even if the model exhibits kernel behavior. It is thus difficult to relate the loss curves in Figure 5 to the evolution of the empirical neural tangent kernel over the course of instruction tuning.

Direct verification is too challenging. Kernel behavior is called as such because the dynamics are described by a fixed $|\mathcal{D}| \times |\mathcal{D}|$ kernel, where each entry contains the inner product of the gradients of the model output with respect to the parameters for different datapoints. Subtly, this matrix must be constructed for every logit, so the computational cost scales with the number of classes in the dataset. Past works that instantiate this kernel have focused on vision tasks with relatively few classes (Arora et al., 2019; Yu et al., 2020a; Wei et al., 2022a) or prompt-based text classification (Malladi et al., 2023), which only considers a label word for each class in simple downstream tasks. On the other hand, instruction tuning requires a $|\mathcal{V}|$ -dimensional vector of logits from the model, where $\mathcal{V}$ is the vocabulary containing, for example, roughly 32,000 tokens. Additionally, each partial context must be treated as its own input, making $|\mathcal{D}|$ the length of each sequence multiplied by the number of sequences in the dataset. Therefore, computing the kernel for instruction tuning is computationally infeasible, given the massive size of $\mathcal{D}$ and the large number of output classes.

G. Comparison to Data Models (Ilyas et al., 2022)

Concurrent work in Engstrom et al. (2024) applies the datamodels framework (Ilyas et al., 2022) to select pre-training data to induce strong performance on target downstream tasks. Fitting a datamodel generally requires repeated training of a model to fit a mapping between the selected data and the resulting accuracy of the trained model, but Park et al. (2023) reframed the problem to simulate this repeated training procedure at a much lower computational cost. Datamodels and its related techniques are inspired by influence functions (Koh & Liang, 2017), which permit counterfactual reasoning over training (e.g., “what would have happened if I omitted this training datapoint?”). On the other hand, the influence formulation in Pruthi et al. (2020) can only counterfactually reason over one optimization step at a time, not the entire training procedure.

In this section, we describe the how the data selection method in Engstrom et al. (2024), which was applied to the pre-training

¹²This was called the “kernel regime” in Jacot et al. (2018) when training the model from a random initialization.

setting, relates to the considerations and observations reported in this paper.

1. **Reframing data selection as optimization:** Both works reframe the data selection problem as a straightforward optimization problem, whereby one chooses data to maximally improve a given metric. This approach is in contrast to prior works that rely on pre-defined notions of useful data (Gururangan et al., 2020; Chen et al., 2023b) or surface-form features (Xie et al., 2023b). Instead, both works can be considered efficient approximations of meta-optimization approaches (Nguyen et al., 2021).
2. **Approximating training dynamics:** Given the considerable computational cost of training LLMs, both works have to approximate the training dynamics of the model in order to efficiently estimate the effect of including a datapoint. LESS relies on the assumption that the first-order approximation of the loss dynamics is accurate, whereas Engstrom et al. (2024) relies on a first-order approximation of how the model output changes during training. The latter is closer to the kernel view described in the previous section, though we note that the output used in datamodels generally involves the margin, which is not well-covered by kernel theory. An implicit assumption of both of these techniques is that the influences of selected datapoints add linearly (i.e., selecting two equally highly scored datapoints will doubly improve the model performance). Theory in Saunshi et al. (2023) provides a start at understanding when this linearity holds, but it may not apply to all settings. For example, in the case where the same datapoint is duplicated in the training set, we don't expect a marked improvement from the inclusion of the duplicate.
3. **Proxy models and selection models:** Both works require performing initial training in a smaller, computationally efficient setup before performing data selection. Engstrom et al. (2024) trained smaller so-called proxy models to fit the datamodel, whereas LESS requires a short warmup training period using the selection model in order to compute the influence in Definition 3.1.
4. **Sequence selection versus token selection:** In §3.2, we describe how selecting sequences instead of individual tokens requires an adjustment to the influence formulation. The token versus sequence consideration also appears in Engstrom et al. (2024), where the method has to average the model output over the entire sequence before using it in the data model. Because Engstrom et al. (2024) considers the pre-training setting, where all input sequences are of the same (maximal) length, we cannot be sure if their proposed methodology circumvents the sequence length bias described in §3.2.

H. Sequence Level Gradients in Instruction Tuning

In Section 3.2, we discussed that sequence level gradients in instruction tuning exhibit odd properties because they are the average of the token gradients in the completion. In particular, Figure 3 demonstrates that across datasets, the gradient norm is strongly anti-correlated with the length of the completion. Similarly, Figure 4 shows that the norm of the update $\|\Gamma\|$ (see Definition 3.1) decays rapidly with length. Therefore, using the straightforward inner product formulation in Section 2 would bias LESS to select short sequences for the training dataset. Intuitively, training on only short sequences would prevent the model from learning to perform long-form generations, as is required for many tasks. Ablation experiments in Table 13 demonstrate that using inner products for LESS results in a dataset with much shorter sequences. Training on this dataset induces markedly worse performance than using the cosine similarity and sometimes worse performance than random selection. Wang et al. (2020) similarly reported that using dot products for data selection in the machine translation setting was less stable than using cosine similarity.

Importantly, this phenomenon is not a failure of the influence formulation in Section 2. Rather, it suggests that we ought to perform token-wise data selection (e.g., only averaging the loss over a few tokens within each sequence), because some gradients within a sequence are helpful and others are not. At the time of writing, such fine-grained selection is computationally intractable for LLMs, but additional approximations and implementation accelerations may make it possible.

H.1. Theoretical View

Theoretically, the relationship between sequence-level gradients and token gradients is quite interesting. Suppose token gradients were equal to a deterministic sequence-level gradient (e.g., determined by topic) plus some i.i.d. Gaussian noise. This is analogous to the widely accepted theoretical model of example gradients being equal to the full batch gradient with additive noise. Then, as the sequence length grows, we would have more independent samples of the token gradient noise,

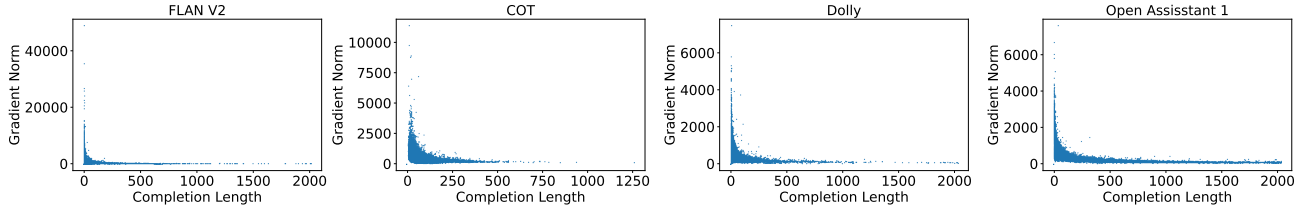


Figure 3: Gradient norms of instruction tuning examples negatively correlates with the length of the completion.

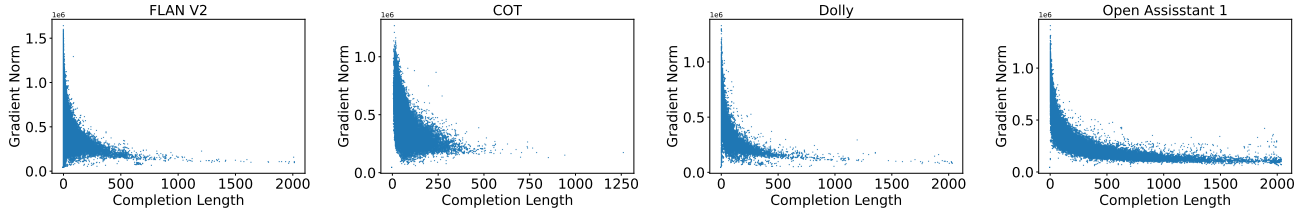


Figure 4: The norm of the update in the Adam optimizer for instruction tuning examples negatively correlates with the length of the completion.

and the average token gradient should concentrate to the deterministic sequence-level gradient. This would yield the trends shown in Figure 3 but it is difficult to believe the gradient noise is i.i.d. because the tokens are not sampled i.i.d. from the corpus. Tokens within a context are semantically related to one another, and constructing a suitable noise model with appropriate correlations that permits analysis appears to be difficult. One possible approach is to assume a generative model for language that assign a latent topic vector to each sequence and generate tokens accordingly, as was done in [Arora et al. \(2016\)](#). Then, the token gradients can be modeled as correlated.

Table 13: Task performance of using normalized gradients and vanilla gradients. We find that using vanilla gradients to select data leads to worse performance than normalized gradients. We also find that the length of the selected data is much shorter when using vanilla gradients, due to the fact that shorter sequences have larger gradient norms (Figures 3 and 4).

	Random		Vanilla Gradients		Normalized Gradients	
	Performance	Length	Performance	Length	Performance	Length
MMLU	46.5	105.4	47.3	11.3	50.2	15.1
TYDIQA	52.7	105.4	52.3	19.7	56.2	44.4
BBH	38.9	105.4	37.4	19.0	41.5	101.2

I. Loss vs Accuracy

Our motivation in Section 2 is to select datapoints that will most dramatically reduce the validation loss. However, for many language modeling tasks, it is unclear how minimizing the cross entropy interacts with the task accuracy. In particular, many complex tasks require the model to generate long-form sequences of text, and it is not clear how well long-form generation capability correlates with accuracy.

In Figures 6 to 8, we provide evidence across model scales and tasks that the minimizing the validation loss does not always correspond to increasing the accuracy. A similar phenomenon was observed in [Zhou et al. \(2023\)](#); [Tay et al. \(2021\)](#). Despite its reliance on the cross entropy loss, LESS boosts accuracy over relevant baselines. Overall, we suggest that optimizing alternative objectives may yield better accuracy, but we leave exploring such ideas to future work. We note that any objective function can be substituted into the derivation in Section 2, so the ideas used in developing LESS are easily extensible.

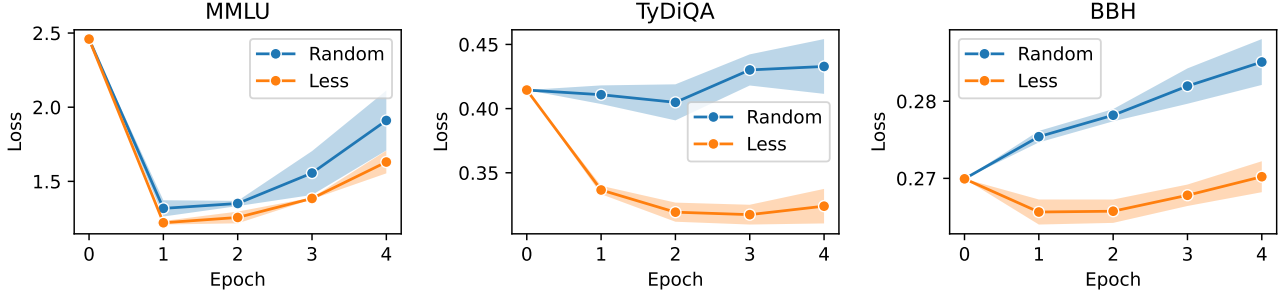


Figure 5: Validation loss of random selection and LESS across checkpoints for LLAMA-2-7B.

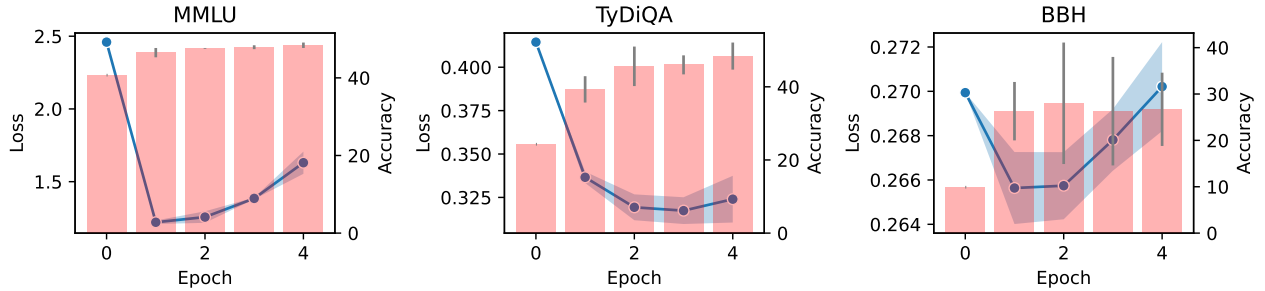


Figure 6: Validation loss (Blue curve) and accuracy (Red bar) across checkpoints for LLAMA-2-7B.

J. Gradient Similarity Analysis

In Figure 9, we plot the gradient similarity between each training example and each validation example in descending order. We observe a subset of examples exhibiting relatively higher gradient similarity scores when compared to the validation data. Following this subset, there is a sharp decline in similarity scores, which then continue to decrease in a very smooth manner. Notably, for MMLU, none of the examples exhibit a negative gradient similarity score. In contrast, for TyDiQA and BBH, there are instances of examples with negative gradient similarity scores. However, we did not observe any obvious patterns in the examples with negative gradient similarity scores. Please find some examples in Appendix K.

K. Qualitative Analysis

K.1. Top Selected Examples for BBH and MMLU

We present more selected examples for MMLU and BBH in Table 18. For MMLU, the examples are drawn from CoT, CoT, and FLAN V2, respectively. The gradient-based approaches return an example of a problem of a similar difficulty level. BM25 also returns a slightly harder math combinations example, where it has token overlap with the target instance (e.g., multiple digit 5). Representation-based approach return a question answering example that does not require math reasoning ability. For BBH, the examples are drawn from CoT, CoT, and DOLLY respectively. The representation and gradient-based approaches capture the essence of the task to judge the truthfulness of a statement, while the BM25 method aligns with superficial ngram features such as NBA Finals.

K.2. Examples with Negative Gradient Similarity Scores

We present a few examples with the lowest gradient similarity scores with the validation data in Table 15 and Table 14. Simply looking at the examples fails to reveal any significant patterns that could explain the observed negative transfer to the target task, which suggests the need for further investigation.

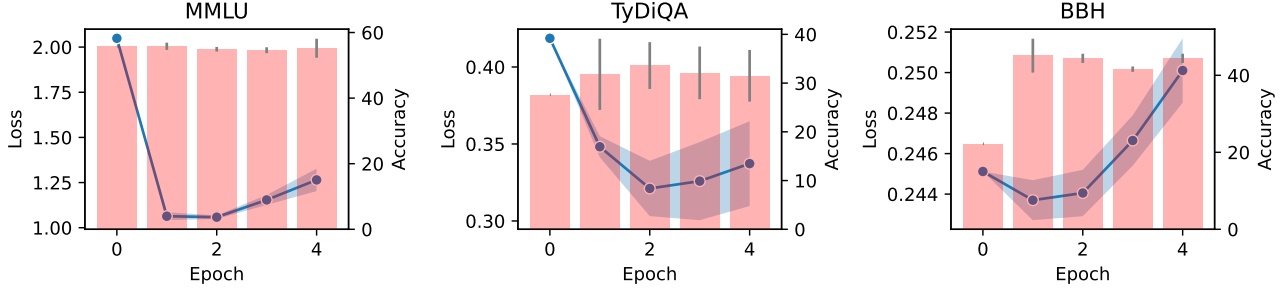


Figure 7: Validation loss (Blue curve) and accuracy (Red bar) across checkpoints for LLAMA-2-13B.

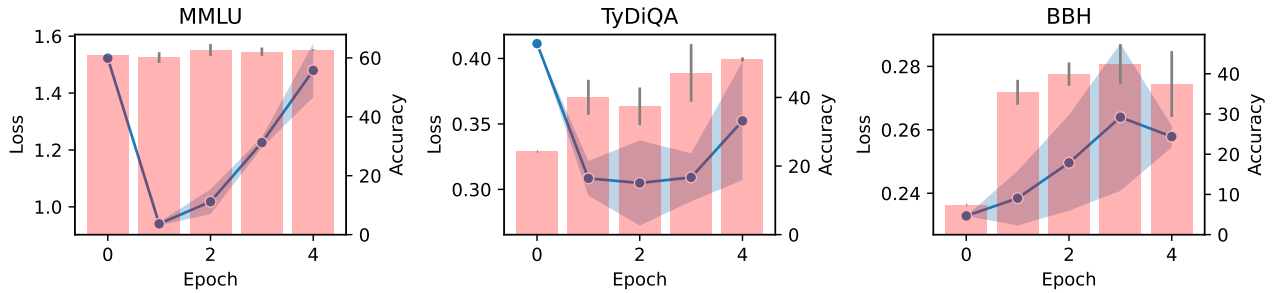


Figure 8: Validation loss (Blue curve) and accuracy (Red bar) across checkpoints for MISTRAL-7B.

K.3. Subset Distribution of Selected Examples

We present the distribution of the selected examples for different methods in Figure 10. Different approaches end up selecting very different amounts of data from each training subset. However, we find that there is not a monotonic relationship between the amount of data selected from a subset and the performance of the method. This suggests that each subset could have useful examples for the target task, and the key is to identify the most relevant examples.

In our specific observations, we notice that LESS predominantly selects data from the FLAN V2 dataset for MMLU. This choice appears intuitive since MMLU predominantly comprises multiple-choice questions. In the case of TYDIQA, LESS exhibits a strong preference for data from the DOLLY dataset. Upon examination, it becomes evident that many of these examples adhere to an open-book question answering format. Finally, for BBH, LESS primarily selects data from the CoT dataset. This preference aligns with the fact that BBH tasks involve chain of thought reasoning, a characteristic prominently present in the CoT dataset’s examples.

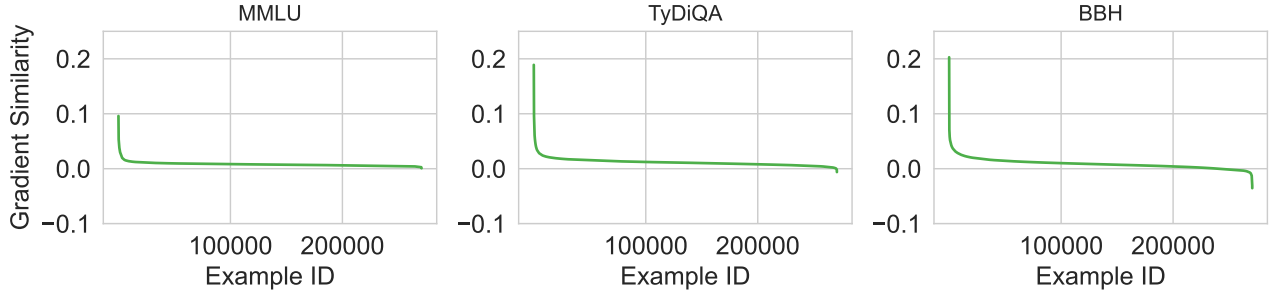


Figure 9: Gradient similarity between training examples and validation examples. The examples are ordered by the gradient similarity score.

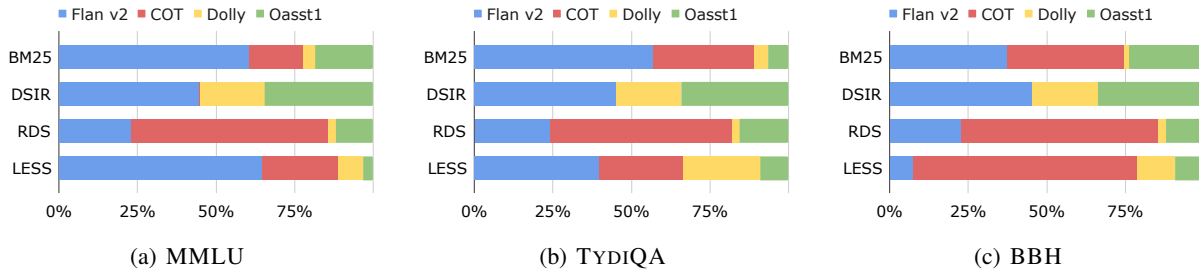


Figure 10: Subset distribution of the top 5% selected examples for different methods.

Table 14: Examples with a negative gradient similarity score to the validation examples for TYDIQA.

TYDIQA validation example
User: The following are multiple choice questions (with answers) about elementary mathematics. The population of the city where Michelle was born is 145,826. What is the value of the 5 in the number 145,826? A. 5 thousands B. 5 hundreds C. 5 tens D. 5 ones Answer: Assistant: The answer is: A
Selected examples with lowest Inf_{Adam} scores
User: Instruction: What would be some useful random items to help you on an island if you were stranded Output: Assistant: A hockey skate would be great because of the blades and laces Glasses could maybe be used to help you start a fire if you used the glasses to reflect the sunlight The cord from a blow dryer could be quite useful User: q: Where could you buy an old pie safe with decorative tin? Options: - shopping bag - trash - antique store - landfill - cupboard a: Old items can be found at an antique store. From an antique store, you could buy an old pie safe with decorative tin. The answer is antique store QUESTION: What article of apparel or accessory is typically made of felt? Options: - shoes - table - hat shop - clothes - man's hat ANSWER: Felt is a type of textile material. Man's hat are generally made of felt material. Hat can be article of apparel or accesory. The answer is man's hat q: A lot of people play the fiddle in what state which comes first alphabetically? Options: - tennessee - violin case - orchestra - alabama - alaska a: Assistant: Lot of people in alabama state also plays fiddle like other states. Alabama comes first alphabetically than other state as it starts from alphabet A. The answer is alabama

Table 15: Examples with a negative gradient similarity score to the validation examples for BBH.

BBH validation example
User: Determine whether an artificially constructed sentence relating to sports is plausible or not. Q: Is the following sentence plausible? "Bam Adebayo scored a reverse layup in the Western Conference Finals." Assistant: A: Let's think step by step. Bam Adebayo is an American basketball player. Scoring a reverse layup in the Western Conference Finals is part of the NBA Finals. So the answer is yes.
Selected examples with lowest Inf_{Adam} scores
User: Ques & Ans: Sam Allardyce has managed the most teams in the Premier League, having taken charge of seven different clubs: Bolton Wanderers, Newcastle United, Blackburn Rovers, West Ham United, Sunderland, Crystal Palace and Everton. Who has managed the most premier league teams? Sam Allardyce Stream of consciousness: To answer this question, we should know that: Sam Allardyce has managed the most teams in the Premier League, having taken charge of seven different clubs: Bolton Wanderers, Newcastle United, Blackburn Rovers, West Ham United, Sunderland, Crystal Palace and Everton. Ques & Ans: "Shout" is a song by British band Tears for Fears, written by Roland Orzabal and Ian Stanley and sung by Orzabal (with Curt Smith duetting on the chorus). First released in the UK on 23 November 1984, it was the band's eighth single release (the second taken from their second album Songs from the Big Chair) and sixth UK Top 40 hit, peaking at no. 4 in January 1985. In the US, it reached no. 1 on the Billboard Hot 100 on 3 August 1985 and remained there for three weeks. "Shout" would become one of the most successful songs of 1985, eventually reaching the Top Ten in 25 countries. "Shout" is regarded as one of the most recognisable songs from the mid-eighties and is also recognised as the group's signature song, along with "Everybody Wants to Rule the World". Answer this question: when did shout by tears for fears come out? 23 November 1984 Stream of consciousness: Assistant: The relevant information is: First released in the UK on 23 November 1984, it was the band's eighth single release (the second taken from their second album Songs from the Big Chair) and sixth UK Top 40 hit, peaking at no.
User: next question: Where would you find the nearest attic? Options: - victorian style house - basement - older houses - own home - victorian house reasoning: Assistant: Own home is the house where one lives. One would find the nearest attic in his house. The answer is own home.

Table 16: Top selected examples when using one TYDIQA example as $\mathcal{D}_{\text{val}}$ for data selection with different approaches. The TYDIQA example is in the context of Bengali question answering. These examples are drawn from FLAN V2, FLAN V2, and DOLLY, respectively. Notably, LESS demonstrates the capability to prioritize examples of the same task type (question answering), whereas the other methods tend to be influenced by the linguistic attributes of the validation example, leading to the selection of Bengali examples that do not align with the same task type. Please find the translation of the table in Table 17.

A TydiQA Validation Example: Question Answering in Bengali		
User: প্রদত্ত অধ্যায়ের তথ্যের উপর ভিত্তি করে নিম্নলিখিত প্রশ্নের উত্তর দিন। অধ্যায়: বাংলার নবজাগরণ ও ব্রাহ্মসমাজ-কেন্দ্রিক সামাজিক-সাংস্কৃতিক সংস্কার আন্দোলন বাংলার সাংস্কৃতিক ও অর্থনৈতিক জীবনে গভীর প্রভাব বিস্তার করে। ১৮৫৭ সালের মহাবিদ্রোহের সূচনা কলকাতার অদূরেই হয়েছিল। এই বিদ্রোহ ব্যর্থ হলেও এর পরিপ্রেক্ষিতে ভারতের শাসনভার কোম্পানির হাত থেকে ব্রিটিশ রাজশক্তি স্বহস্তে গ্রহণ করে। ভারত শাসনের জন্য একটি ভাইসরয়ের পদ সৃষ্টি করা হয়। ১৯০৫ সালে ধর্মীয় প্রশ্ন: অবিভক্ত স্বাধীন বাংলা কবে দিখান্ডিত হয়? Assistant: উত্তর: ১৯০৫ সালে		
Selected by BM25	Selected by RDS	Selected by LESS
<i>Masked Word Prediction in Bengali</i> User: Select the most logical word from four options to replace the <MASK> token in the given Bengali statement. [Q]: Statement: ১৯৬১-৬২ মৌসুমে টেড ডেস্টারের নেতৃত্বাধীন ইংরেজ দলের সদস্যরা পাকিস্তান ও <MASK> সফরে ঐ দুই টেস্টে অংশগ্রহণ করেন। তার বোলিং ভঙ্গীমার কারণে টেস্ট অভিষেকে পর্ব ক্ষণিকটা বিলম্বিত হয়। Option A: গ্ল্যামারগন Option B: লিচেস্টারশায়ারের Option C: ভারত Assistant: [A]: ভারত	<i>Hate Speech Classification in Bengali</i> User: You are given a hateful post in Bengali that expresses hate or encourages violence towards a person or a group based on the protected characteristics such as race, religion, sex, and sexual orientation. You are expected to classify the post into two classes: personal or non-personal depending on the topic. Q: তুমি কি সুখী হতে চাও না? না চাই না কিছু কেন? সুখে থাকলে ভুতে কিল্লায় আমি কিল খেতে চাই না Assistant: personal	<i>Question Answering in English</i> User: Given the question and input, write a response to answer the question. Which year was quantum computer demonstrated to be possible? Input: Over the years, experimentalists have constructed small-scale quantum computers using trapped ions and superconductors. In 1998, a two-qubit quantum computer demonstrated the feasibility of the technology, [...] Response: Assistant: 1998

Table 17: English translations of Table 16.

A TydiQA Validation Example: Question Answering in Bengali (Translated)		
User: Answer the following question based on the content of the given chapter. Chapter: The Bengali Renaissance and the Brahmo Samaj - in a nutshell, the social reformers and the reforms they introduced had a profound impact on the social and economic life of Bengal. The beginning of the great rebellion took place in Calcutta in 1857. After the failure of this rebellion, the British Empire accepted the rule of the East India Company's hands as a friendly power. For the governance of India, the position of a Viceroy was created. In 1905, the religious and political motivations led to the division of Bengal [...] Question: When was Bengal divided? Assistant: Answer: 1905.		
Selected by BM25 (Translated)	Selected by RDS (Translated)	Selected by LESS
<i>Prediction of the Masked Word in Bengali</i> User: Choose the most logical word from four options to replace the <MASK> token in the given Bengali statement. [Q]: Statement: In the 1961-62 season, the touring English team's captain participated in the Pakistan <MASK> sphere for those two tests. His availability for bowling marked the delay of his test debut for some time. Option A: Glamorgan Option B: Leicestershire Option C: India Assistant: [A]: Glamorgan	<i>Hate Speech Classification in Bengali</i> User: You are given a hateful post in Bengali that expresses hate or encourages violence towards a person or a group based on the protected characteristics such as race, religion, sex, and sexual orientation. You are expected to classify the post into two classes: personal or non-personal depending on the topic. Q: Do you not want to be happy alone? Not that I don't want to, but now? When happy, I forget that I don't want to eat alone. Assistant: personal	<i>Question Answering in English</i> User: Given the question and input, write a response to answer the question. Which year was quantum computer demonstrated to be possible? Input: Over the years, experimentalists have constructed small-scale quantum computers using trapped ions and superconductors. In 1998, a two-qubit quantum computer demonstrated the feasibility of the technology, [...] Response: Assistant: 1998

Table 18: Top examples selected for a MMLU validation instance in the context of elementary mathematics and selected for a BBH target instance in the context of sports understanding. We compare examples selected by BM25, RDS and LESS.

MMLU	BBH
A Validation Example	A Validation Example
User: The following are multiple choice questions (with answers) about elementary mathematics. The population of the city where Michelle was born is 145,826. What is the value of the 5 in the number 145,826? A. 5 thousands B. 5 hundreds C. 5 tens D. 5 ones Answer: Assistant: The answer is: A	User: Determine whether an artificially constructed sentence relating to sports is plausible or not. Q: Is the following sentence plausible? "Bam Adebayo scored a reverse layup in the Western Conference Finals." Assistant: A: Let's think step by step. Bam Adebayo is an American basketball player. Scoring a reverse layup in the Western Conference Finals is part of the NBA Finals. So the answer is yes.
Selected by BM25	Selected by BM25
User: Question: A multiple choice test consists of 4 questions, and each question has 5 answer choices. In how many ways can the test be completed if every question is unanswered? Options: (A) 24 (B) 120 (C) 625 (D) 720 (E) 1024 Answer: 5 choices for each of the 4 questions, thus total of $5 * 5 * 5 * 5 = 5^4 = 625$ ways. Assistant: The answer is (C).	User: Give a stream of consciousness and then the final answer. The 2016 NBA Finals was the championship series of the National Basketball Association (NBA) 2015 – 16 season and conclusion of the 2016 playoffs. The Eastern Conference champion Cleveland Cavaliers defeated the Western Conference champion Golden State Warriors 4 – 3 in a rematch of the 2015 NBA Finals. [...] Based on this passage, who did Cleveland play in the finals last year? Assistant: To answer this question, we should know that: The Eastern Conference champion Cleveland Cavaliers defeated the Western Conference champion Golden State Warriors 4 – 3 in a rematch of the 2015 NBA Finals. So, the final answer is Golden State Warriors.
Selected by RDS	Selected by RDS
User: Of course, the most unforgettable classmate I had was also the most vicious bully. I attended a boarding school for badly behaved children and it had its fair share of bullies who had been expelled from mainstream schools. But this particular bully had even the teachers running scared. When he went off on one, it could take up to four teachers or members of care staff to get him under control. [...] At the end of the story, the bully is probably: A. Now a teacher B. Now an article writer C. Deceased from drug abuse D. not enough information Assistant: C	User: Is Y2K relevant to the plot of The Godfather? Please give rationale first, then the answer. Assistant: The story in the Godfather spans from 1945 to 1955. Y2K refers to events related to the formatting and storage of calendar data for dates beginning in the year 2000. The answer is: no.
Selected by LESS	Selected by LESS
User: Q: In this task, you need to provide the correct option for a given problem from the provided options. Problem: the compound ratio of $2/3$, $6/7$, $4/3$ and $1/8$ is given by ? Options: a) $2/21$, b) $1/48$, c) $1/98$, d) $1/96$, e) $1/68$ Assistant: a	User: Please answer the following request: Is the following state true or false: Peanuts are vegetables? Answer: Assistant: This statement is true. Peanuts are part of the legume family and are therefore vegetables.